# Computationally Efficient Predictions for Multi-Output Spatiotemporal Dynamical Systems

Joey Hotz
joey.hotz@mail.utoronto.ca

Supervisor: Dr. Meredith Franklin
Examiners: Dr. Ricardo Baptista & Dr. Joshua Speagle

Department of Statistical Sciences
University of Toronto

October 30, 2026

**Abstract**

Over the past decade, several groups of machine learning researchers have demonstrated promising computational algorithms and mathematical results which utilize Gaussian process (GP) regression and Kalman filtering (KF) to derive estimates and predictions for the future states of spatiotemporal dynamical systems in a more computationally-efficient manner.

This paper builds on previous results in this research area by adapting and extending the results of previous work on univariate outputs towards dynamical systems with vector-valued outputs. In this paper, we introduce the mathematical background necessary to understand Gaussian processes and Kalman filtering, and cover prior results in these areas. We then formally describe the multi-output spatiotemporal dynamics problem, and depict the assumptions required for similar computational speedups to exist in the multi-output spatiotemporal dynamics setting. Lastly, we apply these algorithms to both synthetic and real-world datasets to evaluate their performance.

# Contents


**1 Introduction** **1**

**2 Background** **2**

2.1 Gaussian Processes . . . . . 2

2.2 Multi-Output Gaussian Processes . . . . . 3

2.3 Kalman Filtering . . . . . 7

**3 Problem Formulation** **9**

3.1 Single-Output Dynamical System . . . . . 9

3.2 Adaptive Location Space . . . . . 13

3.3 Multi-Output Dynamical System . . . . . 13

**4 Methods** **16**

4.1 Simulations . . . . . 16

4.2 Results . . . . . 18

**5 Discussion** **19**

5.1 Conclusions . . . . . 19

5.2 Limitations . . . . . 19

5.3 Future Work . . . . . 19

**A AI Usage Statement** **23**

**B Code** **23**

**C Definitions and Technical Details** **24**

**D Figures** **26**

# 1 Introduction

Gaussian processes (GPs) are an 'industry standard' procedure for modelling the values of unknown functions, especially in scenarios where there are noisy or otherwise unreliable measurements and/or correlations in the data which must be accounted for (Rasmussen and Williams, 2006).

Gaussian process regression, also commonly referred to in the literature as *kriging* after the South African geostatistician Danie G. Krige (Cressie, 1990), is a statistical modelling procedure wherein a finite collection of observations are used for learning the form of an unknown real-valued function. Gaussian processes are particularly useful when there are correlations among observations, as these models can leverage information from correlated data to reduce the variance of outputs in the estimated function. Furthermore, the equations for fitting a GP to a dataset have computable closed-form solutions. Due to these convenient properties, Gaussian processes are frequently used in a variety of practical statistical applications involving modelling spatiotemporal data, particularly when we aim to learn certain functional properties of the underlying data.

Although Gaussian processes are mathematically tractable and convenient to use, there is one *major* drawback to these models: exact inference with Gaussian processes has a cubically-scaling computational overhead, as the procedure is bottlenecked by the cost to compute the Cholesky factorization of the covariance matrix. Fitting a Gaussian process onto a dataset which contains $M$ observations is a procedure requiring $\mathcal{O}(M^3)$ floating point operations (Rasmussen and Williams, 2006), making these regression problems infeasible for many large-scale datasets (Guhaniyogi and Banerjee, 2018; Pförtner et al., 2025; Salari and Zeinali, 2025).

As a brief theoretical example of this poor scaling, fitting a Gaussian process on a 'moderately-sized' dataset containing 1000 observations requires $\mathcal{O}(10^9)$ floating point operations. In applied scientific scenarios involving spatiotemporal modelling where there are repeated measurements, the complexity of Gaussian process regression cubically with both of the number of locations *and* the number of measurements per location, which may quickly cause these models to surpass the scope of feasible calculation.

Furthermore, statistical modelling with Gaussian processes requires choosing a spatiotemporal covariance model which yields a positive semidefinite matrix on *any* set of inputs (Rasmussen and Williams, 2006). The class of valid covariance functions for a Gaussian process are mathematically convenient to work with in such regression contexts, but Gaussian process regression does impose further restrictions on how we choose to represent the covariance among observations in a spatiotemporal data setting.

Due to these computational difficulties, many statisticians and machine learning researchers have sought to develop algorithms for fitting a Gaussian process with reduced computational overhead. Prominent well-studied and broadly applicable methods for accomplishing this task include approximations using induced latent variables (Quiñonero-Candela and Rasmussen, 2005), sparse variational Gaussian processes (Titsias, 2009), and Vecchia approximations (Katzfuss and Guinness, 2021). However, these methods all require some degree of approximation, trading the exactness of a solution for increased algorithmic efficiency.

In their 2020 paper *Efficient spatio-temporal Gaussian regression via Kalman filtering*, Todescato et al. introduce a new algorithm for efficiently deriving estimates of the future states in a spatio-temporal dynamical system under certain assumptions for the covariance structure between observations. The algorithm proposed by the authors utilizes Gaussian process regression in tandem with the Kalman filter, providing the ability to account for both the spatial and temporal aspects

which guide the 'evolution' of the dynamical system while simultaneously reducing the overall computational load.

This paper provides a detailed overview of the results in Todescato et al. (2020) and subsequently builds upon the scaffolding in the authors' work by amending their proposed algorithm to operate with vector-valued outputs instead of being confined to scalar responses. This extension to multivariate response vectors provides many practical benefits, as there are many scientific research areas ranging from geostatistics and climatology to fields such as electrical engineering and signal processing (Särkkä and Hartikainen, 2012; Huang et al., 2025) which utilize spatiotemporal models to account for the relationships between correlated response variables.

# 2 Background

In order to properly describe the algorithm devised by Todescato et al. (2020), we must first separately discuss Gaussian processes and Kalman filtering.

## 2.1 Gaussian Processes

A Gaussian process (GP) is a real-valued stochastic process indexed over some $P$-dimensional *state space* $\mathcal{X} \subseteq \mathbb{R}^P$ which satisfies the following key property:

**Definition 2.1** (Gaussian Process)**.** *The real-valued function $f : \mathcal{X} \to \mathbb{R}$ follows a Gaussian process on $\mathcal{X}$ if for every finite collection ($N$-tuple) of states $\{\mathbf{x}_1, \cdots, \mathbf{x}_N\} \subset \mathcal{X}$, the joint distribution of the random vector $\big(f(\mathbf{x}_1), \cdots, f(\mathbf{x}_N)\big)^\mathsf{T}$ is a multivariate Gaussian distribution on $\mathbb{R}^N$.*

Furthermore, every Gaussian process on the state space $\mathcal{X}$ is *uniquely* specified by two real-valued functions: a mean function $m : \mathcal{X} \to \mathbb{R}$ and a *positive-definite* covariance (kernel) function $k : \mathcal{X} \times \mathcal{X} \to \mathbb{R}^{\geq 0}$, which respectively describe the mean and covariance of the underlying process (Rasmussen and Williams, 2006). Any function $f : \mathcal{X} \to \mathbb{R}$ which is a realization of a Gaussian process with mean $m(\cdot)$ and kernel $k(\cdot, \cdot)$ is endowed with the following structure:

$$\textbf{Mean:} \quad m(\mathbf{x}) = \mathbb{E}[f(\mathbf{x})] \text{ for all } \mathbf{x} \in \mathcal{X} \tag{2.1a}$$

$$\textbf{Covariance:} \quad k(\mathbf{x}, \mathbf{x}') = \mathrm{Cov}(f(\mathbf{x}), f(\mathbf{x}')) \text{ for all } (\mathbf{x}, \mathbf{x}') \in \mathcal{X} \times \mathcal{X} \tag{2.1b}$$

We denote a function $f : \mathcal{X} \to \mathbb{R}$ which is drawn from a Gaussian process with mean $m(\cdot)$ and kernel $k(\cdot, \cdot)$ as $f \sim \mathcal{GP}(m, k)$. Note that the kernel function $k(\cdot, \cdot)$ in Equation (2.1b) must be symmetric in its arguments, as this function describes the covariance between the Gaussian process outputs at these two locations. In addition to describing the covariance between function observations, the kernel function $k(\cdot, \cdot)$ serves another complementary purpose, as sample paths have continuous mean-square derivatives up to the order of the kernel $k(\cdot, x)$ (Rasmussen and Williams, 2006).

Suppose that $f \sim \mathcal{GP}(m, k)$ is a realization of some Gaussian process on $\mathcal{X} \subseteq \mathbb{R}^P$, and let $\{\mathbf{x}_1, \ldots, \mathbf{x}_N\} \subset \mathcal{X}$ be a set of queried locations. For ease of notation, we define the data matrix $\mathbf{X} := \big[\, \mathbf{x}_1^\mathsf{T} \cdots \mathbf{x}_N^\mathsf{T} \,\big]^\mathsf{T} \in \mathbb{R}^{N \times P}$, and define the corresponding vectors $f(\mathbf{X}) := \big[\, f(\mathbf{x}_1^\mathsf{T}) \cdots f(\mathbf{x}_N^\mathsf{T}) \,\big]^\mathsf{T} \in \mathbb{R}^N$ and $m(\mathbf{X}) := \big[\, m(\mathbf{x}_1^\mathsf{T}) \cdots m(\mathbf{x}_N^\mathsf{T}) \,\big]^\mathsf{T} \in \mathbb{R}^N$. Lastly, let $k(\mathbf{X}, \mathbf{X}) \in \mathbb{R}^{N \times N}$ represent the *Gram matrix* for

the observed locations which is induced by the kernel $k(\cdot, \cdot)$:

$$k(\mathbf{X}, \mathbf{X}) := \begin{bmatrix} k(\mathbf{x}_1, \mathbf{x}_1) & k(\mathbf{x}_1, \mathbf{x}_2) & \cdots & k(\mathbf{x}_1, \mathbf{x}_N) \\ k(\mathbf{x}_2, \mathbf{x}_1) & k(\mathbf{x}_2, \mathbf{x}_2) & \cdots & k(\mathbf{x}_2, \mathbf{x}_N) \\ \vdots & \vdots & \ddots & \vdots \\ k(\mathbf{x}_N, \mathbf{x}_1) & k(\mathbf{x}_N, \mathbf{x}_2) & \cdots & k(\mathbf{x}_N, \mathbf{x}_N) \end{bmatrix} \tag{2.2}$$

One of the key properties which allows Gaussian processes to be so widely-applicable is that under the conditions described above, the vector $f(\mathbf{X})$ follows a multivariate Gaussian distribution:

$$f(\mathbf{X}) \sim \mathcal{N}\Big(m(\mathbf{X}), k(\mathbf{X}, \mathbf{X})\Big) \tag{2.3}$$

Furthermore, consider the same setting as above, and suppose that we have some (potentially) noisy measured observations given by $y_i = f(\mathbf{x}_i) + \epsilon_i$, for independent normally-distributed measurement errors $\epsilon_1, \ldots, \epsilon_N \overset{\text{i.i.d.}}{\sim} \mathcal{N}(0, \sigma^2)$. Equivalently, we assume that $y_i \sim \mathcal{N}\big(f(\mathbf{x}_i), \sigma^2\big)$ for each queried location $\mathbf{x}_i$ (Rasmussen and Williams, 2006). Defining the response vector $\mathbf{y} := \begin{bmatrix} y_1 & \cdots & y_N \end{bmatrix}^\mathsf{T} \in \mathbb{R}^N$ yields the following distributional result:

$$\mathbf{y} \sim \mathcal{N}\Big(m(\mathbf{X}), k(\mathbf{X}, \mathbf{X}) + \sigma^2 \mathbf{I}_N\Big) \tag{2.4}$$

Gaussian processes are often incorporated in a Bayesian setting as a prior distribution which one can place on the space of real-valued functions on $\mathcal{X}$ (Titsias, 2009; Álvarez et al., 2012). For example, for a *black-box* function $f : \mathcal{X} \to \mathbb{R}$, we may place a prior of the form $f \sim \mathcal{GP}(m, k)$, for specified functions $m : \mathcal{X} \to \mathbb{R}$ and $k : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ as described by Equations (2.1a) and (2.1b). If observations of the black-box function $f(\cdot)$ are taken with normally-distributed measurement error (as described by Equation (2.4)), then the posterior predictive distribution for $f(\cdot)$ will be a Gaussian process whose mean and kernel are altered by conditioning on the observed data. In particular, for a data matrix $\mathbf{X} \in \mathbb{R}^{N \times P}$ with the response vector $\mathbf{y} \in \mathbb{R}^N$, the posterior predictive distribution for $f$ evaluated at any points $\mathbf{x}, \mathbf{x}' \in \mathcal{X}$ is given by the following:

$$\textbf{Mean:} \quad m_{\text{post}}(\mathbf{x}) = m(\mathbf{x}) + k(\mathbf{x}, \mathbf{X})\big[k(\mathbf{X}, \mathbf{X}) + \sigma^2 \mathbf{I}_N\big]^{-1}\big(\mathbf{y} - m(\mathbf{X})\big) \tag{2.5a}$$

$$\textbf{Kernel:} \quad k_{\text{post}}(\mathbf{x}, \mathbf{x}') = k(\mathbf{x}, \mathbf{x}') - k(\mathbf{x}, \mathbf{X})\big[k(\mathbf{X}, \mathbf{X}) + \sigma^2 \mathbf{I}_N\big]^{-1} k(\mathbf{X}, \mathbf{x}') \tag{2.5b}$$

In the equations above, $k(\mathbf{x}, \mathbf{X})$ represents the 1-by-$N$ Gram matrix with entries $k(\mathbf{x}, \mathbf{x}_i)$ and similarly, $k(\mathbf{X}, \mathbf{x}')$ is the $N$-by-1 Gram matrix with entries $k(\mathbf{x}_i, \mathbf{x}')$. Setting $\mathbf{x} = \mathbf{x}'$ in Equation (2.5b) yields the posterior predictive variance for $f(\mathbf{x})$. Furthermore, when the observations $y_1, \ldots, y_N$ are assumed to be exact (i.e. $\sigma^2 = 0$), then Equations (2.5a) and (2.5b) will perfectly interpolate every observation. To be precise, if $\sigma^2 = 0$, the posterior Gaussian process satisfies that $m_{\text{post}}(\mathbf{x}_i) = y_i$ and $k_{\text{post}}(\mathbf{x}_i, \mathbf{x}_i) = 0$, implying that each $f(\mathbf{x}_i)$ is *exactly* equal to the corresponding $y_i$.

## 2.2 Multi-Output Gaussian Processes

Thus far, we have focused solely on the description of single-output Gaussian processes, whose realizations are described by functions of the form $f : \mathcal{X} \to \mathbb{R}$. However, there is a natural extension from these univariate processes to *multi-output* (also called *multi-task*) Gaussian processes: a higher-dimensional form of a Gaussian process, whose outputs are vectors in $\mathbb{R}^D$ (Álvarez et al.,

2012; Bonilla et al., 2007).

**Definition 2.2** (Multi-Output Gaussian Process)**.** *The vector-valued function* $\mathbf{f} = (f_1, \ldots, f_D)^\mathsf{T} : \mathcal{X} \to \mathbb{R}^D$ *follows a multi-output Gaussian process on* $\mathcal{X}$ *if for every finite collection of states* $\{\mathbf{x}_1, \ldots, \mathbf{x}_N\} \subset \mathcal{X}$*, the joint distribution of the* $ND$ *random variables* $\big(f_1(\mathbf{x}_1), \cdots, f_D(\mathbf{x}_N)\big)^\mathsf{T}$ *is a multivariate Gaussian distribution on* $\mathbb{R}^{ND}$*.*

Multi-output GPs (MOGPs) are naturally applicable whenever there are several (potentially) correlated quantities which are measured over some common domain. These processes include both *auto-correlations* within the same output and *cross-correlations* between outputs, allowing for information about one output to reduce the uncertainty of other response variables in the model. Conceptually, a multi-output GP is analogous to simultaneously modelling $D$ single-output Gaussian processes, while also modelling the covariance between these processes.

Given states $\mathbf{X} = \big[\mathbf{x}_1^\mathsf{T} \cdots \mathbf{x}_N^\mathsf{T}\big]^\mathsf{T} \in \mathbb{R}^{N \times P}$, we define the matrix $\mathbf{f}(\mathbf{X}) \in \mathbb{R}^{N \times D}$ by concatenating the vectorized columns, i.e. $\mathbf{f}(\mathbf{X}) = \big[f_1(\mathbf{X}) \cdots f_D(\mathbf{X})\big]$, and we can analogously define the matrix $\mathbf{m}(\mathbf{X}) \in \mathbb{R}^{N \times D}$ as a concatenation of the respective $N$-dimensional mean vectors $m_1(\mathbf{X}), \ldots, m_D(\mathbf{X})$. In order to work with a multi-output GP, we must also construct a Gram matrix $\mathbf{K}(\mathbf{X}, \mathbf{X}) \in \mathbb{R}^{ND \times ND}$ which defines the covariance structure between both the queried locations and the output dimensions. In its most general form, the Gram matrix providing the covariance for a multi-output GP has the following structure:

$$\mathbf{K}(\mathbf{X}, \mathbf{X}) := \begin{bmatrix} \mathbf{K}(\mathbf{x}_1, \mathbf{x}_1) & \mathbf{K}(\mathbf{x}_1, \mathbf{x}_2) & \cdots & \mathbf{K}(\mathbf{x}_1, \mathbf{x}_N) \\ \mathbf{K}(\mathbf{x}_2, \mathbf{x}_1) & \mathbf{K}(\mathbf{x}_2, \mathbf{x}_2) & \cdots & \mathbf{K}(\mathbf{x}_2, \mathbf{x}_N) \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{K}(\mathbf{x}_N, \mathbf{x}_1) & \mathbf{K}(\mathbf{x}_N, \mathbf{x}_2) & \cdots & \mathbf{K}(\mathbf{x}_N, \mathbf{x}_N) \end{bmatrix} \tag{2.6}$$

The covariance matrix in Equation (2.6) is overall quite similar to the Gram matrix in the univariate setting which is shown in Equation (2.2). However, the multi-output Gram matrix consists of $N^2$ covariance matrix sub-blocks $\mathbf{K}(\mathbf{x}_i, \mathbf{x}_j) = \mathrm{Cov}(\mathbf{f}(\mathbf{x}_i), \mathbf{f}(\mathbf{x}_j))$ of size $D \times D$, each of which has entries of the form $[\mathbf{K}(\mathbf{x}_i, \mathbf{x}_j)]_{(d,d')} = \mathrm{Cov}(f_d(\mathbf{x}_i), f_{d'}(\mathbf{x}_j))$ (Álvarez et al., 2012). Vectorizing the matrix $\mathbf{f}(\mathbf{X})$ yields the following distributional result:

$$\mathrm{vec}\big(\mathbf{f}(\mathbf{X})\big) \sim \mathcal{N}\Big(\mathrm{vec}\big(\mathbf{m}(\mathbf{X})\big),\, \mathbf{K}(\mathbf{X}, \mathbf{X})\Big) \tag{2.7}$$

Additionally, suppose that we have observations of the form $\mathbf{y}_i = \mathbf{f}(\mathbf{x}_i) + \boldsymbol{\epsilon}_i$ with independent measurement error vectors $\boldsymbol{\epsilon}_1, \ldots, \boldsymbol{\epsilon}_N \overset{\text{i.i.d.}}{\sim} \mathcal{N}(\mathbf{0}, \boldsymbol{\Sigma}_\epsilon)$, for some positive-definite covariance matrix $\boldsymbol{\Sigma}_\epsilon \in \mathbb{R}^{D \times D}$. The joint distribution of these observed values is given by

$$\mathrm{vec}\big(\mathbf{f}(\mathbf{X})\big) \sim \mathcal{N}\Big(\mathrm{vec}\big(\mathbf{m}(\mathbf{X})\big),\, \mathbf{K}(\mathbf{X}, \mathbf{X}) + \mathbf{I}_N \otimes \boldsymbol{\Sigma}_\epsilon\Big) \tag{2.8}$$

Again, the full distribution in Equation (2.8) closely echoes the univariate result from Equation (2.4), with a slight adjustment to account for covariances between outputs.

From a purely theoretical standpoint, multi-output Gaussian processes are defined quite analogously to the single-output processes discussed in Section 2.1. However, by solely introducing the theory and notation regarding MOGPs, we have obscured the major practical implications of these models; namely, the extreme cost for computational inference. In the (general) single-output Gaussian process regime, the cost of inference scales cubically with the size of the matrix

$k(\mathbf{X}, \mathbf{X})$, giving a computational cost of the order $\mathcal{O}(N^3)$. In the setting where the responses are vector-valued, statistical inference still requires the inversion of the Gram matrix $\mathbf{K}(\mathbf{X}, \mathbf{X})$, whose cost balloons to the order of $\mathcal{O}(N^3 D^3)$. Even when $D \ll N$, this is a significant increase in the computational requirement for an already-expensive operation. We will now turn our attention towards three specific cases of multi-output Gaussian processes, which give simpler forms for the Gram matrix $\mathbf{K}(\mathbf{X}, \mathbf{X})$.

**Fully Independent Model:** The simplest form of a multi-output Gaussian process is one in which each output dimension is assumed to be an independent univariate Gaussian process: $f_d \sim \mathcal{GP}(m_d, k_d)$. Under this assumption, the Gram matrix $\mathbf{k}(\mathbf{X}, \mathbf{X})$ is a block-diagonal matrix with $D$ $N \times N$ sub-blocks. This model is computationally quite cheap, as it can be simply decoupled into $D$ separate single-output regression problems. However, this model makes the strong assumption that the targeted outputs are fully independent, and it shares no information across the output dimensions, which hampers its applicability in many practical contexts.

This model is computationally cheap, since it decouples into $D$ separate single-output problems, but it shares no information across outputs whatsoever. The models below reintroduce dependencies between the outputs by expressing them as the linear combinations of latent Gaussian processes.

**Linear Model of Coregionalization (LMC):** The LMC model for a Gaussian process with a vector-valued output is predicated on the assumption that the output functions are built from a linear combination of some set of independent mean-zero univariate Gaussian processes (Álvarez et al., 2012). Suppose that the latent Gaussian processes $u_1, \ldots, u_R$ are independent, with $\mathbb{E}[u_i(\mathbf{x})] = 0$ and $\text{Cov}(u_r(\mathbf{x}), u_{r'}(\mathbf{x}')) = \mathbf{1}_{\{r=r'\}} \times k_r(\mathbf{x}, \mathbf{x}')$, so that the covariance is nonzero if and only if $r = r'$.

Following the description given by Álvarez et al. (2012), we can sort these latent GPs based on their covariance function, giving us $Q$ groups of size $\{R_q\}_{q=1}^{Q}$. By sorting the latent processes in this manner, each group $q$ (for $q = 1, \ldots, Q$) consists of $R_q$ independent univariate Gaussian processes of the form $u_q^1, \ldots, u_q^{R_q} \overset{\text{i.i.d.}}{\sim} \mathcal{GP}(0, K^{(q)})$. Note that even though the covariance structures are shared for latent functions in the same group, the respective Gaussian processes are still entirely independent. As such, $\text{Cov}(u_q^i(\mathbf{x}), u_{q'}^{i'}(\mathbf{x}')) = \mathbf{1}_{\{q=q'\}} \times \mathbf{1}_{\{i=i'\}} \times K^{(q)}(\mathbf{x}, \mathbf{x}')$ under this regrouping of the latent functions.

In the LMC model, we assume that the desired functions $\{f_d(\cdot)\}_{d=1}^{D}$ for the multivariate process are constructed via a linear combination of the latent Gaussian processes $u_1, \ldots, u_R$. Each function is therefore described by the following latent model:

$$f_d(\mathbf{x}) = m_d(\mathbf{x}) + \sum_{r=1}^{R} \alpha_{d,r} u_r(\mathbf{x}) \tag{2.9}$$

Grouping the latent functions by their kernel, this can be written in the form

$$f_d(\mathbf{x}) = m_d(\mathbf{x}) + \sum_{q=1}^{Q} \sum_{i=1}^{R_q} \alpha_{d,q}^i u_q^i(\mathbf{x}) \tag{2.10}$$

The functions $\{m_d(\cdot)\}_{d=1}^{D}$ in Equations (2.9) and (2.10) ensure that each function in the observable multi-output process can have a non-zero mean. Using the latent form given in Equation (2.10),

we can represent the covariance between two outputs by the following:

$$\operatorname{Cov}(f_d(\mathbf{x}), f_{d'}(\mathbf{x}')) = \sum_{q=1}^{Q}\sum_{q'=1}^{Q}\sum_{i=1}^{R_q}\sum_{i=1}^{R_{q'}} \alpha_{d,q}^{i}\alpha_{d',q'}^{i'} \operatorname{Cov}\left(u_q^i(\mathbf{x}), u_{q'}^{i'}(\mathbf{x})\right) \tag{2.11a}$$

In the summation above, the covariance is non-zero if and only if $q = q'$ and $i = i'$.

$$= \sum_{q=1}^{Q}\sum_{i=1}^{R_q} \alpha_{d,q}^{i}\alpha_{d',q}^{i} \operatorname{Cov}\left(u_q^i(\mathbf{x}), u_q^i(\mathbf{x})\right) \tag{2.11b}$$

$$= \sum_{q=1}^{Q}\sum_{i=1}^{R_q} \alpha_{d,q}^{i}\alpha_{d',q}^{i} K^{(q)}(\mathbf{x}, \mathbf{x}') \tag{2.11c}$$

For each group $q = 1, \ldots, Q$, and each pair of dimensions $(d, d')$, we define the value $b_{d,d'}^{(q)} := \sum_{i=1}^{R_q} \alpha_{d,q}^{i}\alpha_{d',q}^{i}$. We can then concatenate these values into a $D \times D$ *coregionalization* matrix $\mathbf{B}^{(q)}$ such that the $(d, d')$ entry of $\mathbf{B}^{(q)} = b_{d,d'}^{(q)}$. Using these coregionalization matrices yields the kernel function for the linear model of coregionalization, which is given by

$$\mathbf{K}(\mathbf{x}, \mathbf{x}') = \sum_{q=1}^{Q} K^{(q)}(\mathbf{x}, \mathbf{x}')\mathbf{B}^{(q)}, \quad \text{with } \mathbf{K}(\mathbf{x}, \mathbf{x}') \in \mathbb{R}^{D\times D}, \tag{2.12}$$

and the overall Gram matrix for the linear model of coregionalization is given by

$$\mathbf{K}(\mathbf{X}, \mathbf{X}) = \sum_{q=1}^{Q} K^{(q)}(\mathbf{X}, \mathbf{X}) \otimes \mathbf{B}^{(q)}, \quad \text{with } \mathbf{K}(\mathbf{X}, \mathbf{X}) \in \mathbb{R}^{ND\times ND} \tag{2.13}$$

The linear model of coregionalization is commonly used for describing the kernels of multi-output Gaussian processes in scientific modelling settings such as inference among sensor networks, geostatistical applications, and robot dynamics (Álvarez et al., 2012; Liu et al., 2018).

**Intrinsic Coregionalization Model (ICM):** The intrinsic coregionalization model (ICM) for a multi-output Gaussian process is a special case of the LMC kernel, where the number of distinct kernel functions ($Q$) is equal to one. The ICM kernel is a more restrictive simplification of the LMC kernel, but the Gram matrix will be significantly less expensive to decompose as a result of this simplifying assumption.

The ICM kernel can be derived in the same format as the LMC kernel, with all latent functions $u_1, \ldots, u_R \overset{\text{i.i.d.}}{\sim} \mathcal{GP}(0, k)$. The covariances generated by the ICM kernel are described by the following relations:

$$\operatorname{Cov}(f_d(\mathbf{x}), f_{d'}(\mathbf{x}')) = k(\mathbf{x}, \mathbf{x}') \times \mathbf{B}(d, d') \tag{2.14}$$

The covariance given in Equation (2.14) corresponds to a kernel which is similar to that in Equation (2.13), with only one spatial kernel $k(\mathbf{X}, \mathbf{X}) \in \mathbb{R}^{N\times N}$ and one corresponding coregionalization matrix $\mathbf{B} \in \mathbb{R}^{D\times D}$ for the outputs.

$$\mathbf{K}(\mathbf{X}, \mathbf{X}) = k(\mathbf{X}, \mathbf{X}) \otimes \mathbf{B}, \quad \text{with } \mathbf{K}(\mathbf{X}, \mathbf{X}) \in \mathbb{R}^{ND\times ND} \tag{2.15}$$

The ICM kernel is an example of a *separable* kernel, as the Gram matrix in 2.15 can be written

as the Kronecker product of one purely spatial Gram matrix and another purely output-related Gram matrix. Similarly, the individual terms of the form $k(\mathbf{X}, \mathbf{X}) \otimes \mathbf{B}^{(q)}$ in the summation in 2.13 are separable kernels.

**Semiparametric Latent Factor Model (SLFM):** Another special case of the LMC kernel is the *semiparametric latent factor model* (SLFM) (Bonilla et al., 2007), in which there is exactly one kernel corresponding to each group, i.e. $R_q = 1$ for each $q \in \{1, \ldots, Q\}$ in the equation in Equation (2.10). Under this scenario, each matrix $\mathbf{B}^{(q)}$ is a rank-one matrix generated by the coefficients $\alpha_{d,q}$, which also allows for the resulting matrix decomposed in a less expensive manner than the traditional LMC kernel.

**Other Constructions:** Many valid MOGP kernels do not satisfy the sum of separable kernels form which was depicted above, such as *process convolution* approaches (Boyle and Frean, 2004) and other invariant forms of kernels which incorporate lags or spatial shifts (Álvarez et al., 2012). In this paper, we focus on multi-output kernels which can be represented via a sum of separable kernels, as the Kronecker product structure combines naturally with the separability requirements considered by Todescato et al. (2020). This produces a convenient structure for the Gram matrices which we consider in Section 3.

## 2.3 Kalman Filtering

The Kalman filter (KF), introduced and eponymously named for Rudolf E. Kálmán (1960), is a technique for recursively estimating the unobserved (latent) states of a linear dynamical system from a sequence of noisy and indirect measurements of the system. Kálmán originally derived the filter as the means to solving an optimal linear estimation problem via orthogonal projections. However, the Kalman filter also lends itself to a statistical interpretation, as the filter is equivalent to the solution of a Bayesian inference problem for a *linear Gaussian state-space model* whose posterior distribution is updated sequentially as new observations are recorded (Särkkä and Svensson, 2023). In this subsection of the paper, we will formally describe the Kalman filter, and the linear Gaussian state-space model which underpins the statistical viewpoint for the Kalman filter. These statistical properties of interest are central to the algorithm presented by Todescato et al. (2020).

**Definition 2.3** (State Space Model)**.** *Let $t_0 < t_1 < t_2 < \cdots < t_J$ be an indexed sequence of time points. A state space model is a dynamical system describing the joint behaviour of two stochastic processes: a latent state process $\{\mathbf{s}_j\}_{j=0}^{J}$ for states $\mathbf{s}_j \in \mathbb{R}^n$ which evolves over time according to some Markovian dynamics, and an observation (or measurement) process $\{\mathbf{y}_j\}_{j=1}^{J}$, where each measurement $\mathbf{y}_j \in \mathbb{R}^m$ is a (potentially) noisy observation of some deterministic function of the current latent state $\mathbf{s}_j$.*

One particularly convenient form of a state space model is the *linear Gaussian* state space model. In this setting, the dynamics which guide the evolution of the system over time and the observation model are both linear functions of the underlying states, and the random terms in the model follow multivariate Gaussian distributions.

**Definition 2.4** (Linear Gaussian State Space Model)**.** *A linear Gaussian state space model is a dynamical state space model whose latent state and observation model are described by the following*

*equations:*

$$\textbf{Initial state:} \quad \mathbf{s}(0) \sim \mathcal{N}\big(\mathbf{m}(0), \boldsymbol{\Sigma}(0)\big) \tag{2.16a}$$

$$\textbf{Dynamic model:} \quad \mathbf{s}(j+1) = \mathbf{A}(j)\mathbf{s}(j) + \mathbf{w}(j), \qquad \mathbf{w}(j) \sim \mathcal{N}\big(\mathbf{0}, \mathbf{Q}(j)\big) \tag{2.16b}$$

$$\textbf{Observation model:} \quad \mathbf{y}(j) = \mathbf{C}(j)\mathbf{s}(j) + \mathbf{v}(j), \qquad \mathbf{v}(j) \sim \mathcal{N}\big(\mathbf{0}, \mathbf{R}(j)\big) \tag{2.16c}$$

In the definition above, $\{\mathbf{s}(j)\}_{j=0}^{J}$ are the latent states within the dynamical model. The evolution of the dynamical system over time is guided by the *state transition matrix* $\mathbf{A}(j)$ and the random Gaussian noise $\mathbf{w}(j)$, and the observation process is influenced by the *observation matrix* $\mathbf{C}(j)$ and observation noise $\mathbf{v}(j)$. The initial state $\mathbf{s}(0)$, process noise terms $\{\mathbf{w}(j)\}_{j=0}^{J-1}$ and observation noise terms $\{\mathbf{v}(j)\}_{j=1}^{J}$ are all assumed to be mutually independent. Furthermore, the parameters $\mathbf{m}(0), \boldsymbol{\Sigma}(0), \{\mathbf{Q}(j)\}_{j=0}^{J-1}$, and $\{\mathbf{R}(j)\}_{j=1}^{J}$, and the transition and observation matrices are assumed to have fixed, known values.

A key property of the multivariate Gaussian distribution is that *any* linear transformation – including affine linear transformations – of a Gaussian-distributed random vector will still follow a Gaussian distribution, as does the sum of any pair of *independent* Gaussian-distributed vectors. Consequently, in the linear Gaussian state space model described by Definition 2.4, every latent state $\{\mathbf{s}(j)\}_{j=0}^{J}$ and measurement $\{\mathbf{y}(j)\}_{j=1}^{J}$ is a random vector drawn from a Gaussian distribution with mathematically tractable parameters (Särkkä and Svensson, 2023).

In such cases, the standard method for recovering the parameters describing the distribution of each random vector is to utilize the Kalman filter (Kálmán, 1960), which iteratively computes the respective conditional mean and covariance for each latent state and observation. This is accomplished by iterating between two steps; the *prediction* step and the *update* step.

**Algorithm 1** The Kalman Filter (Kálmán, 1960)

**Require:**
$\widehat{\mathbf{m}}(0 \mid 0)$: expected value of the initial state $\mathbf{s}(0)$
$\boldsymbol{\Sigma}(0 \mid 0)$: variance of the initial latent state $\mathbf{s}(0)$

**for** $j = 1$ to $J$ **do**
`# Prediction Step:`
  $\widehat{\mathbf{m}}(j \mid j-1) = \mathbf{A}(j-1)\widehat{\mathbf{m}}(j-1 \mid j-1)$
  $\boldsymbol{\Sigma}(j \mid j-1) = \mathbf{A}(j-1)\boldsymbol{\Sigma}(j-1 \mid j-1)\mathbf{A}(j-1)^{\mathsf{T}} + \mathbf{Q}(j-1)$
`# Update Step:`
  $\mathbf{d}(j) = \mathbf{y}(j) - \mathbf{C}(j)\widehat{\mathbf{m}}(j \mid j-1)$
  $\mathbf{S}(j) = \mathbf{C}(j)\boldsymbol{\Sigma}(j \mid j-1)\mathbf{C}(j)^{\mathsf{T}} + \mathbf{R}(j)$
  $\mathbf{K}(j) = \boldsymbol{\Sigma}(j \mid j-1)\mathbf{C}(j)^{\mathsf{T}}\mathbf{S}(j)^{-1}$ ▷ Kalman gain
  $\widehat{\mathbf{m}}(j \mid j) = \widehat{\mathbf{m}}(j \mid j-1) + \mathbf{K}(j)\mathbf{d}(j)$
  $\boldsymbol{\Sigma}(j \mid j) = \boldsymbol{\Sigma}(j \mid j-1) - \mathbf{K}(j)\mathbf{S}(j)\mathbf{K}(j)^{\mathsf{T}}$
**end for**

The prediction and update steps in the Kalman filtering algorithm (Algorithm 1) are so named as they correspond to specific conditional distributions of $\mathbf{s}(j)$ and $\mathbf{y}(j)$ before and after conditioning on the latest observation. In particular, for the linear Gaussian state space model described by Definition 2.4, we have the following distributional relationships:

$$\mathbf{s}(j) \mid \mathbf{y}(1), \ldots, \mathbf{y}(j-1) \sim \mathcal{N}\big(\widehat{\mathbf{m}}(j \mid j), \widehat{\boldsymbol{\Sigma}}(j)\big) \tag{2.17a}$$

$$\mathbf{y}(j) \mid \mathbf{y}(1), \ldots, \mathbf{y}(j-1) \sim \mathcal{N}\big(\mathbf{C}(j)\widehat{\mathbf{m}}(j), \mathbf{S}(j)\big) \tag{2.17b}$$

$$\mathbf{s}(j) \mid \mathbf{y}(1), \ldots, \mathbf{y}(j) \sim \mathcal{N}\big(\mathbf{m}(j), \boldsymbol{\Sigma}(j)\big) \tag{2.17c}$$

Each of the $J$ iterations of Algorithm 1 requires both multiplying and inverting matrices. In total, the computational complexity per iteration in the Kalman filtering loop is $\mathcal{O}(m^3+n^3)$ (Raitoharju and Piché, 2019), with each of these steps occurring $J$ times in the total run of the algorithm. Thus, a 'naive' implementation of this algorithm would have a total computational complexity on the order of $\mathcal{O}\big(J(m^3+n^3)\big)$. Overall, the computational complexity of the Kalman filtering algorithm is *linear* with respect to the number of iterations $J$, which is far quicker than the cubically-scaling $\mathcal{O}(J^3)$ runtime required for Gaussian process regression on a set of $J$ observations. This algorithm also has a much lower storage requirement, as the updates are based strictly on the previous state, allowing for overwriting of earlier estimators.

# 3 Problem Formulation

Now that we have discussed the pre-requisite information regarding Gaussian process regression and Kalman filtering, we may begin to describe the setting considered by Todescato et al..

First, we will describe the initial univariate Gaussian process regression problem posed by the authors, and the algorithm which they propose as a means of more efficiently deriving their desired estimates. Following this, we will explain how the algorithm by Todescato et al. can be extended for use with multi-output spatiotemporal dynamical systems, while maintaining similar computational benefits in comparison to related MOGP-based methods.

## 3.1 Single-Output Dynamical System

Suppose that we have some *compact* state space $\mathcal{X} \subseteq \mathbb{R}^P$ which represents a set of all possible locations of interest, such as the latitudes and longitudes of a bounded geographical region, or the coordinates of pixels in an image. Within the state space $\mathcal{X}$, we have a *finite* set of "observable" locations $\mathcal{I} := \{x_1, \ldots, x_M\} \subset \mathcal{X}$.

We wish to perform inference on the values of some function $f : \mathcal{X} \times \mathbb{R}^+ \to \mathbb{R}$, which we assume to be a realization of some zero-mean Gaussian process $f \sim \mathcal{GP}(0, K)$, for some kernel function $K$. Furthermore, at times $t_0 < t_1 < \ldots$, we record noisy observations of the form

$$y_i(t_j) = f(x_i, t_j) + v_i(t_j), \quad v_i(t_j) \sim \mathcal{N}(0, \sigma^2) \tag{3.1}$$

Importantly, the set of observations which are recorded at each time instant $t_j$ may differ. For each time instant $t_j$, we let $\mathcal{M}_j \subseteq \mathcal{I}$ represent the observations recorded at time $t_j$, with $M_j := |\mathcal{M}_j|$.

The ultimate goal of the algorithm proposed by Todescato et al. is to develop estimates of the *entire* function $f(x,t)$ at any $x \in \mathcal{X}$ and $t \in \mathbb{R}^+$ based solely on statistical inference on observations from the set of pre-specified locations $\mathcal{I} \subset \mathcal{X}$.

**Assumption 3.1.** *The kernel function $K$ representing the covariance of the univariate GP $f(x,t)$ can be separated into the product of a temporal and spatial kernel,*

$$K((x,t),(x',t')) = K_s(x,x')\gamma(\tau), \quad \textit{where } \tau := |t-t'| \tag{3.2}$$

**Assumption 3.2.** *The power spectral density $S(\omega)$ (see Definition C.7) of $\gamma(|t-t'|)$ can be written as a rational function (with respect to $\omega$) of order $2r$.*

The separability requirement enforced by Assumption 3.1 restricts the class of kernel functions which can be used for describing the covariance structure of the Gaussian process. However, this assumption is not overly restrictive in practice, and the family of kernels meeting this criterion remains quite flexible. Similarly, Assumption 3.2 does not excessively confine the options for temporal kernels in the Gaussian process, as many commonly-used kernels such as the Matérn covariance family satisfy this requirement of the power spectrum (Rasmussen and Williams, 2006).

In order to apply the Kalman filter, the algorithm proposed by Todescato et al. relies on a discretized state-space representation of the dynamical process which guides the evolution of the function $f(x,t)$. In many applied settings, the dynamical system which is desired evolves according to a continuous-time process. However, the measurement model is based on discrete-time observations. To overcome this distinction, the authors introduce a discretized state-space representation of the underlying continuous-time dynamics which guide the 'true' growth of the stochastic dynamical system of interest.

For any time instance $t_j$ where measurements are recorded, let $\tau_j := t_j - t_{j-1}$. Additionally, let the three matrices $F \in \mathbb{R}^{r\times r}, G \in \mathbb{R}^{r\times 1}$ and $H \in \mathbb{R}^{1\times r}$ be the model matrices which collectively form a state-space representation for the spectral density $S(\omega)$, as described by Carron et al. and Theorem C.3.

**Definition 3.1** (Companion Form Matrices)**.** *Suppose $S(\omega)$ is a rational power spectral decomposition for the covariance function $\gamma(\tau)$ with order $2r$. Let $S(\omega)$ have the spectral factorization (see Theorem C.3) $S(\omega) = W(\boldsymbol{i}\omega)W(-\boldsymbol{i}\omega)$, where*

$$W(\boldsymbol{i}\omega) := \frac{b_{r-1}(\boldsymbol{i}\omega)^{r-1} + b_{r-2}(\boldsymbol{i}\omega)^{r-2} + \cdots + b_0}{(\boldsymbol{i}\omega)^r + a_{r-1}(\boldsymbol{i}\omega)^{r-1} + \cdots + a_0}, \tag{3.3}$$

*as in Equation (C.9). The companion form model matrices corresponding to the PSD $S(\omega)$ are the following:*

$$F = \begin{bmatrix} 0 & 1 & 0 & \cdots & 0 \\ 0 & 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & 1 \\ -a_0 & -a_1 & -a_2 & \cdots & -a_{r-1} \end{bmatrix} \in \mathbb{R}^{r\times r} \tag{3.4a}$$

$$G = \begin{bmatrix} 0 & 0 & 0 & \cdots & 1 \end{bmatrix}^\intercal \in \mathbb{R}^{r\times 1} \tag{3.4b}$$

$$H = \begin{bmatrix} b_0 & b_1 & b_2 & \cdots & b_{r-1} \end{bmatrix} \in \mathbb{R}^{1\times r} \tag{3.4c}$$

Let $\overline{K}_s \in \mathbb{R}^{M\times M}$ be the Gram matrix for the spatial kernel $K_s$ applied to $\mathcal{I}$. The spatiotemporal dynamics for the desired system are discretized by the following pair of equations, which guide the evolution from one observation time to the next:

$$\mathbf{s}(j+1) = A(j)\mathbf{s}(j) + \mathbf{w}(j), \quad \mathbf{w}(j) \sim \mathcal{N}(\mathbf{0}, Q(j)) \tag{3.5a}$$

$$\mathbf{y}(j) = C(j)\mathbf{s}(j) + \mathbf{v}(j), \quad \mathbf{v}(j) \sim \mathcal{N}(0, R(j)) \tag{3.5b}$$

The Markovian dynamics which impact Equations (3.5a) and (3.5b) are parametrized by the following matrices. Formally, the four matrices given below are indexed by the observation time $t_j$, but the only manner in which they differ across observation times is through changes to the

location set $\mathcal{M}_j \subset \mathcal{I}$. If the locations are always identical (i.e. $\mathcal{M}_1 = \cdots = \mathcal{M}_J = \mathcal{I}$), then these matrices are fully constant.

$$A(j) = e^{\tau_j(\mathbf{I}_M \otimes F)} \in \mathbb{R}^{rM \times rM} \tag{3.6a}$$

$$C(j) = I_j \overline{K}_s^{1/2} (\mathbf{I}_M \otimes H) \in \mathbb{R}^{M_j \times rM} \tag{3.6b}$$

$$Q(j) = \mathbf{I}_M \otimes \left( \int_0^{\tau_j} \left(e^{\tau F}\right) G G^{\mathsf{T}} \left(e^{\tau F}\right) \mathrm{d}\tau \right) \in \mathbb{R}^{rM \times rM} \tag{3.6c}$$

$$R(j) = \sigma^2 \mathbf{I}_{M_j} \in \mathbb{R}^{M_j \times M_j} \tag{3.6d}$$

The matrix $I_j$ in Equation (3.6b) is an indicator matrix $I_j \in \{0,1\}^{M_j \times M}$ which selects for the subset of $x_i \in \mathcal{M}_j$ from the entire location set $\mathcal{I}$.

Given the definitions above, we now present a pseudocode implementation of the primary Kalman-based Gaussian process regression algorithm suggested by Todescato et al.. This algorithm has two inputs: a desired estimation time $t \in \mathbb{R}^+$, and a dataset $\mathcal{D}$ containing observations of the form $y_i(t_j) = f(x_i, t_j) + v_i(t_j)$, as described by Equation (3.1).

**Algorithm 2** Kalman-based GP regression on $\mathcal{I}$ at query time $t \in \mathbb{R}^+$

**Require:**

$(F, G, H)$: state-space representation of $S_r(\omega)$
$\sigma^2$: measurement noise variance
$\mathcal{I}$: set of valid input locations
$K_s(\cdot,\cdot)$: spatial kernel function
$\gamma(\cdot)$: temporal kernel function
$\mathbf{\Sigma}_0$: solution of Lyapunov equation $FX + XF^{\mathsf{T}} + GG^{\mathsf{T}} = 0$

Initialize $\widehat{\mathbf{s}}(0 \mid 0) = \mathbf{0}$ and $\mathbf{\Sigma}(0 \mid 0) = \mathbf{I} \otimes \mathbf{\Sigma}_0$
Set $j \leftarrow 0$
`Kalman estimates:`
**while** $j \leq J$ and $t_{j+1} \leq t$ **do**
    Compute $A(j), C(j), Q(j), R(j)$ using Equations (3.6a) to (3.6d)
    Apply the Kalman filter (Algorithm 1) to compute $\widehat{\mathbf{s}}(j+1 \mid j+1)$ and $\mathbf{\Sigma}(j+1 \mid j+1)$
    $j \leftarrow j + 1$
**end while**
`Open-loop prediction:`
$\tau = t - t_j$
$\widehat{\mathbf{s}}(t) = e^{\tau(\mathbf{I} \otimes F)} \widehat{\mathbf{s}}(j \mid j)$
$\mathbf{\Sigma}^{\mathbf{s}}(t) = \left(e^{\tau(\mathbf{I} \otimes F)}\right) \mathbf{\Sigma}(j|j) \left(e^{\tau(\mathbf{I} \otimes F)}\right)^{\mathsf{T}}$
`Process Estimate:`
$\widehat{\mathbf{f}}(t) = \overline{K}_s^{1/2} (\mathbf{I} \otimes H) \widehat{\mathbf{s}}(t)$
$\mathbf{\Sigma}^{\mathbf{f}}(t) = \overline{K}_s^{1/2} (\mathbf{I} \otimes H) \mathbf{\Sigma}^{\mathbf{s}}(t) (\mathbf{I} \otimes H)^{\mathsf{T}} \overline{K}_s^{1/2}$
**return** $\widehat{\mathbf{f}}(t)$ and $\mathbf{\Sigma}^{\mathbf{f}}(t)$

Algorithm 2 is a cleaned-up representation of the primary Kalman-based Gaussian process regression algorithm presented by Todescato et al. in their paper, with some alterations to make the pseudocode simpler to understand. Instead of presenting the algorithm as a continuous, non-terminating function which 'loops' over all $t \in \mathbb{R}^+$, we have altered the formulation of the algorithm to explicitly demonstrate the Kalman filtering aspect, and the subsequent estimation of the change in the process post-filtering.

The loop in this algorithm iteratively performs Kalman filtering updates to the mean and

covariance until one of the two loop conditions is falsified; either $t_j \leq t < t_{j+1}$ or $t > t_J$, and there are no further observations to feed into the Kalman filtering loop in the algorithm. Once this loop condition is broken, the algorithm makes an 'open-loop' prediction of the state at time $t$ using the matrices $F, G, H$ from the state-space representation to describe the continuous-time dynamics affecting further changes to the state from time $t_j$ to time $t$. These predictions are then passed through the Gaussian process estimation

Though it is not an inherent part of the algorithm, in practice, Algorithm 2 should be treated as an *online* algorithm, with the most recent Kalman-filtered state maintained in memory until a new set of observations are recorded and transmitted. Storing these terms in local memory will both allow for the estimation of current-time measurements based on a continuous-time stochastic dynamical equation, and it prevents the need to recompute previous states when subsequent measurements are transmitted and the updates to the current estimated states are desired.

As part of their paper, Todescato et al. describe and prove a *Dynamic Representer Theorem* Theorem 3.1; a form of the classic 'Representer Theorem' for Gaussian processes (see Theorem C.1) which is amenable to the presumed spatiotemporal dynamics in their model. The key practical result of their Dynamic Representer Theorem is that for any time $t$ beyond the final observation time $t_J$, the mean and variance estimates returned by Algorithm 2 can be used to derive a minimum-variance estimator of the function $f(x,t)$ at *any* location $x \in \mathcal{X}$.

**Theorem 3.1** (Dynamic Representer Theorem)**.** *Suppose that we have a Gaussian process $f : \mathcal{X} \times \mathbb{R}^+ \to \mathbb{R}$ with a spatiotemporal kernel $k(\cdot,\cdot)$ which satisfies Assumptions 3.1 and 3.2, and we have some observations recorded at times $t_0, \ldots, t_J$. For any $x \in \mathcal{X}$ and $t \geq t_J$, our estimate of $f(x,t)$ is the following:*

$$\widehat{f}(x,t) = \overline{K}_s(x,\mathcal{I})\overline{K}_s(\mathcal{I},\mathcal{I})\widehat{\mathbf{f}}(t), \tag{3.7}$$

*and the variance of this estimator is given by*

$$\operatorname{Var}\left(\widehat{f}(x,t) \mid \mathcal{D}\right) = \gamma(0)\left[\overline{K}_s(x,x) - \overline{K}_s(x,\mathcal{I})\overline{K}_s(\mathcal{I},\mathcal{I})^{-1}\left(\overline{K}_s(\mathcal{I},\mathcal{I}) - \boldsymbol{\Sigma}^{\mathbf{f}}(t)\right)\overline{K}_s(\mathcal{I},\mathcal{I})^{-1}\overline{K}_s(\mathcal{I},x)\right] \tag{3.8}$$

The key concept behind the authors' proof of Theorem 3.1 is that the assumptions regarding the evolution of the spatiotemporal dynamical system yield a strong conditional independence result. In particular, the value of $f(x,t)$ is *conditionally independent* of the state of the dynamical system at any other time given the observations at time $t$. More formally, for any location $x \in \mathcal{X}$ and measurement time $t \in \mathbb{R}$, we have that

$$f(x,t) \perp\!\!\!\perp \mathbf{f}(\tau) \mid \mathbf{f}(t), \quad \tau \neq t \tag{3.9}$$

The implication of Equation (3.9) is clear for any $x \in \mathcal{I}_j$ when $t = t_j$, as we would therefore already have an observation corresponding to that location at that time. However, the key takeaway of this result is that the mean $\widehat{\mathbf{f}}(t)$ and covariance $\boldsymbol{\Sigma}^{\mathbf{f}}(t)$ given by the filtering distribution in Algorithm 2 can be used as a posterior Gaussian process in estimating the value of $f(x,t)$ at *any* location. As a corollary of this conditional independence result, the Representer Theorem Theorem C.1 for Gaussian processes implies that $\widehat{f}(x,t)$ is a minimum-variance estimator for the value of $f(x,t)$.

In addition to defining and proving the Dynamic Representer Theorem, the authors describe the computational complexity of updating the estimated mean and variance in accordance with Algorithm 2. One iteration of the Kalman filtering algorithm to update the estimated mean and covariance from $\widehat{\mathbf{f}}(t_{j-1})$ and $\boldsymbol{\Sigma}^{\mathbf{f}}(t_{j-1})$ to $\widehat{\mathbf{f}}(t_j)$ and $\boldsymbol{\Sigma}^{\mathbf{f}}(t_j)$ has a computational complexity on the

order of $\mathcal{O}(rMM_j + rM_j^3)$, with an additional cost of $\mathcal{O}(MP)$ to compute predictions of the form $\{f(x_p, t_j)\}_{i=1}^P$. Again, we reiterate here that this computation is made more efficient by treating Algorithm 2 as an online algorithm, with some aspects of the model saved in persistent storage. For instance, extending the estimates to $P$ arbitrary locations requires computing $\overline{K}_s(\mathcal{I},\mathcal{I})^{-1}\overline{K}_s(\mathcal{I}, x_p)$ at each prediction location $x_p$. This matrix inversion and multiplication would typically be of the order $\mathcal{O}(M^2P)$, but the matrix $\overline{K}_s(\mathcal{I},\mathcal{I})^{-1} \in \mathbb{R}^{M\times M}$ is constant across all iterations, so pre-computing and storing this matrix will allow for a significant practical speedup.

## 3.2 Adaptive Location Space

In addition to the presentation of their primary Kalman-enhanced Gaussian process regression algorithm, Todescato et al. also describe a second algorithm which allows for the input location space $\mathcal{I}$ to be adaptively changed over the course of the study, so that we have input location sets $\mathcal{I}(1), \ldots, \mathcal{I}(J)$, instead of a pre-fixed set $\mathcal{I}$ representing all possible measurement locations.

This version of the algorithm lends itself toward many practical applications, as this allows for the measurement equipment to be moved from one location to another without invalidating some of the authors' earlier assumptions. The canonical example given by Todescato et al. is a group of aerial vehicles which measure meteorological data near clouds, but there are many other situations where adaptive input locations may be desired, e.g. researchers choosing where to collect measurements based on some heuristic decision-making system.

If there is some adaptive strategy for choosing input locations, the cardinality of the set $\mathcal{I}(1) \cup \mathcal{I}(2) \cup \cdots \cup \mathcal{I}(j)$ representing all input locations up to time $t_j$ can grow without bound as $j$ increases. This leads to two issues which could arise with Algorithm 2; the amount of memory needed to store previous estimates may grow unboundedly and the Kalman filter needs to be re-run for *every* location added to the set of valid input locations. Both of these issues are significant detriments to this algorithm which greatly inflate the memory requirement for storing past results and the runtime of the entirety of the algorithm.

In their paper, Todescato et al. propose an alteration to Algorithm 2 which circumvents these issues in the adaptive location setting by updating the filter at newly-added points using Algorithm 2 and shedding certain input locations to reduce the ongoing memory in use. The purpose of this alternate algorithm is to avoid exceeding some fixed memory allocation budget. The ability to implement an algorithm which adaptively removes unwanted points without significantly biasing or impacting the overall model is another significant advantage of the Kalman-based GP regression method which could not be fulfilled when performing a 'pure' Gaussian process regression.

## 3.3 Multi-Output Dynamical System

Now, we may focus on the adaptation of the Kalman-enhanced regression algorithm for a multi-output Gaussian process. In addition to the two previously-stated assumptions on the nature of the dynamical system, we must make one additional assumption regarding the observations in the dynamical model.

**Assumption 3.3** (Missingness of Vector-Valued Observations)**.** *For any location $\mathbf{x}_i \in \mathcal{X}$ and time $t_k \in \mathbb{R}^+$, the $D$ entries of the response vector $\mathbf{y}_{i,j} = f(\mathbf{x}_i, t_j) + \mathbf{v}_{i,j} \in \mathbb{R}^D$ will either all be recorded, or all be missing.*

Assumption 3.3 is a simplifying assumption which ensures that there are not any partially-recorded or partially-unrecorded vectors of information. Such an assumption was absent from the prior paper on this topic, as their algorithm focused solely on scalar-valued responses, so it would not be possible for the measurement $\mathbf{y}_i(t_j)$ to be partially observed.

The overall observation model for the multi-output Gaussian process version of the spatiotemporal dynamical system is quite similar to the observation model in Equation (3.1) proposed by Todescato et al. for the univariate case. For any point $x_i \in \mathcal{I}$ and measurement time $t_j$, we model the observed function value as a noisy observation of the underlying process, with

$$\mathbf{y}_i(t_j) = f(x_i, t_j) + \mathbf{v}_i(t_j), \quad \mathbf{v}_i(t_j) \sim \mathcal{N}(\mathbf{0}, \mathbf{V}), \tag{3.10}$$

where $\mathbf{V} \in \mathbb{R}^{D \times D}$ is a fixed, known, covariance matrix for the observational noise.

Our extension of Algorithm 2 focuses on two key changes, which are intertwined with one another. The primary alteration which we make to the authors' provided algorithm is to extend their methodology so that we can derive estimators for the current state in the multi-output process and the corresponding multivariate dynamical system. In addition to this change, our algorithm also relaxes Assumption 3.1 by allowing the 'full' kernel $K$ to be a sum of $Q$ multi-output kernels which are themselves time-separable, as in the LMC construction in Equation (2.13). Suppose that the kernel function $K$ can be written as an LMC kernel of the form

$$K((x,t,d),(x',t',d')) = \sum_{q=1}^{Q} \gamma^{(q)}(|t-t'|) K_s^{(q)}(x,x') \mathbf{B}^{(q)}(d,d'), \tag{3.11}$$

where each temporal covariance function $\gamma^{(1)}(\cdot) \ldots, \gamma^{(Q)}(\cdot)$ individually satisfies Assumption 3.2, with the corresponding power spectrum $S_q(\omega)$ is of the order $2r_q$. For ease of notation, we define $R_Q := \sum_{q=1}^{Q} r_q$ to be the sum of the orders for the $Q$ individual temporal kernels. We define our model matrices $\overline{F} \in \mathbb{R}^{R_Q \times R_Q}$, $\overline{G} \in \mathbb{R}^{R_Q \times Q}$, and $\{ \, widetildeH_q \}_{q=1}^{Q} \in \mathbb{R}^{1 \times R_Q}$ for this set of temporal kernels by the following:

$$\overline{F} = \begin{bmatrix} F_1 & \mathbf{0} & \cdots & \mathbf{0} \\ \mathbf{0} & F_2 & \cdots & \mathbf{0} \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{0} & \mathbf{0} & \cdots & F_Q \end{bmatrix} \in \mathbb{R}^{R_Q \times R_Q} \tag{3.12a}$$

$$\overline{G} = \begin{bmatrix} G_1 & \mathbf{0} & \cdots & \mathbf{0} \\ \mathbf{0} & G_2 & \cdots & \mathbf{0} \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{0} & \mathbf{0} & \cdots & G_Q \end{bmatrix} \in \mathbb{R}^{R_Q \times Q} \tag{3.12b}$$

$$\widetilde{H}_q = \begin{bmatrix} \mathbf{0} & \cdots & H_q & \cdots & \mathbf{0} \end{bmatrix} \in \mathbb{R}^{1 \times R_Q} \tag{3.12c}$$

For the matrices $\widetilde{H}_q$ in Equation (3.12c), all entries are zero aside from the $q^{\text{th}}$ block in the row matrix, which is equal to the companion matrix $H$ for the temporal kernel $\gamma^{(q)}$ from Equation (3.4c). Using these newly-defined companion matrices, we can rewrite the discretization of the state space

model which defines the Kalman filter.

$$A(j) = e^{\tau_j(\mathbf{I}_{MD}\otimes F)} \in \mathbb{R}^{MDR_Q\times MDR_Q} \tag{3.13a}$$

$$C(j) = I_j \sum_{q=1}^{Q} \left[\overline{K}_s^{(q)}\right]^{1/2}(\mathbf{I}_{MD}\otimes \widetilde{H}_q) \in \mathbb{R}^{M_jD\times MDR_Q} \tag{3.13b}$$

$$Q(j) = \mathbf{I}_{MD} \otimes \left(\int_0^{\tau_j} \left(e^{\tau\overline{F}}\right)\overline{G}\,\overline{G}^{\mathsf{T}}\left(e^{\tau\overline{F}}\right)\mathrm{d}\tau\right) \in \mathbb{R}^{MDR_Q\times MDR_Q} \tag{3.13c}$$

$$R(j) = \mathbf{I}_{M_j}\otimes \mathbf{V} \in \mathbb{R}^{M_jD\times M_jD} \tag{3.13d}$$

The four matrices above are interpreted in the same manner as those in Equations (3.6a) to (3.6d) and serve the same purpose in guiding the evolution of the dynamical system over time. However, the dimensions of these matrices have grown significantly, due to both the increase in the number of outputs and by taking a direct sum over all of the individual temporal kernels.

Using the model and Kalman filtering matrices, we are ready to define our Kalman-enhanced MOGP regression algorithm.

**Algorithm 3** Kalman-based MOGP regression on $\mathcal{I}$ at query time $t \in \mathbb{R}^+$

**Require:**

$\{(F_q, G_q, H_q)\}_{q=1}^{Q}$: state-space representation(s) for $S_{r,q}(\omega)$
$\mathbf{V} \in \mathbb{R}^{D\times D}$: measurement noise covariance matrix
$\mathcal{I}$: set of valid input locations
$\{K_s^{(q)}(\cdot,\cdot)\}_{q=1}^{Q}$: spatial kernel function(s)
$\{\gamma^{(q)}(\cdot)\}_{q=1}^{Q}$: temporal kernel function(s)
$\mathbf{\Sigma}_0$: solution of Lyapunov equation $\overline{F}X + X\overline{F}^{\mathsf{T}} + \overline{G}\,\overline{G}^{\mathsf{T}} = 0$

Create $\overline{F}, \overline{G}, \{\widetilde{H}_q\}_{q=1}^{Q}$ per Equations (3.12a) to (3.12c)
Initialize $\widehat{\mathbf{s}}(0 \mid 0) = \mathbf{0}$ and $\mathbf{\Sigma}(0 \mid 0) = \mathbf{I}\otimes\mathbf{\Sigma}_0$
Set $j \leftarrow 0$
`Kalman estimates:`
**while** $j \le J$ and $t_{j+1} \le t$ **do**
    Compute $A(j), C(j), Q(j), R(j)$ using Equations (3.13a) to (3.13d)
    Apply the Kalman filter (Algorithm 1) to compute $\widehat{\mathbf{s}}(j+1 \mid j+1)$ and $\mathbf{\Sigma}(j+1 \mid j+1)$
    $j \leftarrow j+1$
**end while**
`Open-loop prediction:`
$\tau = t - t_j$
$\widehat{\mathbf{s}}(t) = e^{\tau(\mathbf{I}\otimes\overline{F})}\widehat{\mathbf{s}}(j \mid j)$
$\mathbf{\Sigma}^{\mathbf{s}}(t) = \left(e^{\tau(\mathbf{I}\otimes\overline{F})}\right)\mathbf{\Sigma}(j|j)\left(e^{\tau(\mathbf{I}\otimes\overline{F})}\right)^{\mathsf{T}}$
`Process Estimate:`
$\widehat{\mathbf{f}}(t) = \overline{K}_s^{1/2}(\mathbf{I}\otimes H)\widehat{\mathbf{s}}(t)$
$\mathbf{\Sigma}^{\mathbf{f}}(t) = \overline{K}_s^{1/2}(\mathbf{I}\otimes H)\mathbf{\Sigma}^{\mathbf{s}}(t)(\mathbf{I}\otimes H)^{\mathsf{T}}\,\overline{K}_s^{1/2}$
**return** $\widehat{\mathbf{f}}(t)$ and $\mathbf{\Sigma}^{\mathbf{f}}(t)$

For the sake of consistency, Algorithm 3 is presented under the assumption that the spatial kernel is an LMC kernel formed by a linear combination of $Q$ latent Gaussian processes. As above, the ICM can be viewed as a special case of the LMC kernel which arises when $Q = 1$. This algorithm is quite similar to algorithm 2, which is ideal, as the algorithm given by Todescato et al. corresponds to a special case of Algorithm 3 where $Q = 1$ and $D = 1$.

# 4 Methods

A major theoretical benefit of the algorithm proposed by Todescato et al. – and our multi-output extension in Algorithm 3 – is that under the necessary assumptions presented above, these algorithms provide minimum variance estimators with a significant reduction in the required computational resources. In order to determine if this goal is met in a practical setting, it is necessary to assess the efficacy of the proposed algorithm based on testable, reproducible results.

To evaluate the performance of Algorithm 3, we compare the accuracy and speed of Algorithm 1 to a 'naive' multi-output Gaussian process regression algorithm. The code for comparing these algorithms are created in Python, and utilize the `GPyTorch` library (Gardner et al., 2018) for the implementation of computational tasks involving Gaussian processes.

## 4.1 Simulations

The focus of our practical simulation study is to compare Algorithm 3 to a 'traditional' multi-output Gaussian process regression method. To accomplish this, we generate synthetic data according to a multi-output Gaussian process model, and compare the predictive accuracy and overall runtime of these algorithms using a set of four multi-output Gaussian process kernels. The four kernels which are compared are described by the following equations:

$$\textbf{Independent:} \quad K((x,t,d),(x',t',d')) = \gamma(|t-t'|)K_s(x,x')\sigma_d^2\mathbf{1}_{\{d=d'\}} \tag{4.1a}$$

$$\textbf{ICM:} \quad K((x,t,d),(x',t',d')) = \gamma(|t-t'|)K_s(x,x')\mathbf{B}(d,d') \tag{4.1b}$$

$$\textbf{LMC:} \quad K((x,t,d),(x',t',d')) = \gamma(|t-t'|)\sum_{q=1}^{Q} K_s^{(q)}(x,x')\mathbf{B}^{(q)}(d,d') \tag{4.1c}$$

$$\textbf{Temporal LMC:} \quad K((x,t,d),(x',t',d')) = \sum_{q=1}^{Q} \gamma^{(q)}(|t-t'|)K_s^{(q)}(x,x')\mathbf{B}^{(q)}(d,d') \tag{4.1d}$$

These four kernels describe similar, but distinct, covariance structures for a multi-output Gaussian process. The first two kernels are fully separable into the product of covariances across the temporal, spatial, and output dimensions, whereas the two LMC kernels are constructed via a sum of separable kernels, but they are not themselves separable.

$$\textbf{Independent:} \quad K((x,t,d),(x',t',d')) = \exp(-|t-t'|)K_{\text{RBF}}(x,x')\sigma_d^2\mathbf{1}_{\{d=d'\}} \tag{4.2a}$$

$$\textbf{ICM:} \quad K((x,t,d),(x',t',d')) = \exp(-|t-t'|)K_{\text{RBF}}(x,x')\mathbf{B}(d,d') \tag{4.2b}$$

$$\textbf{LMC:} \quad K((x,t,d),(x',t',d')) = \exp(-|t-t'|)\sum_{q=1}^{Q} C_{7/2-q}(x,x')\mathbf{B}^{(q)}(d,d') \tag{4.2c}$$

$$\textbf{Temporal LMC:} \quad K((x,t,d),(x',t',d')) = \sum_{q=1}^{Q} \exp\Big(-\frac{|t-t'|}{q}\Big)C_{7/2-q}(x,x')\mathbf{B}^{(q)}(d,d') \tag{4.2d}$$

The MOGP kernels shown in Equations (4.2a) and (4.2b) are created by taking the product of a Gaussian RBF kernel over the spatial variables with a Matérn(1/2) kernel for the temporal variables, and multiplying by the corresponding matrix entry for the dimensions $(d, d')$. All spatial

and temporal kernels in the study have unit variance and unit length-scale, with one exception: the $q^{\text{th}}$ temporal covariance in the 'temporal LMC' setting (Equation (4.2d)) has a length-scale of $q$.

The kernels in Equations (4.2c) and (4.2d) are created by taking the sum of three Matérn kernels with different parameters ($\nu = \{1/2, 3/2, 5/2\}$) for the spatial covariance, and multiplying these by an associated kernel for the temporal covariance.

The simulation study was carried out on a computing server hosted by the Department of Statistical Sciences at the University of Toronto. The code for the simulations is based on the MATLAB code by Carron et al. and Todescato et al., which is available in a public GitHub repository hosted by the research group including the authors of these two papers. To effectively run the simulations, we used `Python 3.12.5` (Python Software Foundation, 2024) with significant usage of the `gpytorch` (Gardner et al., 2018) library to more easily work with the Gaussian process models.

**Simulation Design:** For the simulations, synthetic data were generated from a zero-mean multi-output Gaussian process at 150 evenly-spaced observation locations $\{x_1, \ldots, x_{150}\}$ on a one-dimensional spatial grid. Each observation $x_i$ was measured at a total of 21 evenly-spaced time instances $t \in \{0, 0.5, \ldots, 10\}$.

For each collection of simulation settings, 80% of the locations were chosen uniformly at random to have available measurements, while the remaining 20% did not have available measurements, and were used solely for prediction. Each measurement from the multi-output Gaussian process has some added *independent* Gaussian random noise, all of which were generated according to the distribution $\mathbf{v}_i(t_j) \overset{\text{i.i.d.}}{\sim} \mathcal{N}(\mathbf{0}, \mathbf{I}/4)$.

For the independent kernel, the output variances were drawn as $\sigma_d^2 \sim \text{Unif}(0.5, 1.5)$. For the ICM, the coregionalization matrix was drawn as $\mathbf{B} = \mathbf{A}\mathbf{A}^\top + \kappa\mathbf{I}$, where $\mathbf{A} \in \mathbb{R}^{D \times D}$ has standard normal entries and $\kappa = 10^{-4}$. Both LMC kernels use $Q = 3$ latent processes, each with a rank-one coregionalization matrix $\mathbf{B}^{(q)} = \mathbf{a}_q\mathbf{a}_q^\top + 0.05\,\mathbf{I}$.

To study the effects of incomplete datasets on the reliability of the estimates, a proportion $\rho$ of the collected observations $\mathbf{y}(x_i, t_j)$ were chosen uniformly to be removed from the dataset. The vectors selected to be dropped by this procedure are missing completely at random (MCAR), with no quota given on the number of observations which may be dropped per observation location or measurement time.

The simulation study implements a full-factorial design over three factors; the MOGP kernel function (the four kernels in Equation (4.2)), the dimensionality of the outputs ($D \in \{1, 2, 3, 5\}$), and the proportion of deleted observations ($\rho \in \{0, 0.2, 0.5\}$), for a total of 48 simulation settings. For each of these 48 settings, we generated 20 independent replicate datasets, giving a total of 960 simulated datasets.

**Estimators and Evaluation:** For each of the 960 simulated datasets in the simulation study, we fitted and compared two estimators. The first estimator is the batch multi-output GP, which conditions on all of the available measurements, and is the standard estimator for a multi-output Gaussian process. The second is the functional form given by the output of Algorithm 3, which uses the Kalman filter to sequentially process observations.

In both settings, we assume full knowledge of the true kernel function and noise covariance matrix which were used in generating the observations. As such, there is no optimization or

estimation over unknown hyperparameters in either of these algorithms. In each simulation, we focus on two values; *Estimation* and *Prediction*.

In this setting, estimation refers to the denoising which tries to recover the latent state corresponding to locations in the observation group, while prediction focuses on estimating the true state at the held-out locations. For each of these methods, we look at the root mean squared error (RMSE) and mean absolute error (MAE) of the estimates from the estimates compared to the true latent states $f(x_i, t_j)$ over all possible locations and observation times. In addition to these two accuracy metrics, we also examine the empirical coverage of the 95% credible intervals derived from the posterior mean and variance estimates. Lastly, we compare the wall-clock runtime of these algorithms to get a sense for how these methods scale with the dimensionality of the inputs.

## 4.2 Results

Here, we present our results for the simulation study described in the previous subsection. In the interest of saving space within the main body of the document, the plots are attached in Section D in the Appendix.

The plots in Figures 1 and 2 generally show quite similar results. In all 48 simulation settings for the simulation study, the MOGP regression method seems to exhibit a slightly better overall fit than the Kalman-based method from Algorithm 3. The discrepancy between these two methods appears to grow with the output dimension $D$, and it also seems to get wider for larger values of the missingness proportion $\rho$. This result, while somewhat disheartening, is to be expected, as these plots compare the RMSE and MAE for these methods across *all* points $(x_i, t_j) \in \mathcal{X} \times \mathbb{R}^+$, not solely for the measurements corresponding to the final observation time. By the structure of the respective methodologies, the fitted MOGP should minimize the squared error loss over the entire dataset $\mathcal{D}$, while the Kalman-based method seeks to best learn the latent space representation at the time $t_J$, which may bias or impact the estimates from the earlier timestamps. For all of these simulations, the prediction error (both MAE and RMSE) is significantly larger than the corresponding error for estimation. Again, this is a fairly reasonable result, as the performance on the training set typically outperforms a hold-out or test dataset, but this is a much larger discrepancy than the actual difference between the two methods.

As shown in Figure 3, the 95% credible intervals created by both of these methods are quite reliable, with the error bars for these empirical proportions almost-always covering the desired 95% threshold. This again signals that the two algorithms perform roughly similarly, and both are able to accomplish the desired task well, even if the MOGP may have a slightly superior performance.

Lastly, we will look at the wall-clock times in Figure 4. Overall, the estimation algorithms are quite similar in wall-clock time, with the MOGP taking slightly longer than the Kalman filtering method as the output dimension $D$ gets larger. This is reasonable, as the computational complexity of MOGP regression exhibits poorer scaling with respect to $D$ than that of Kalman filtering, so one may expect to see similar results arise as $D$ gets larger. However, the difference in wall-clock times for these two methods is extraordinarily large for the predictions on held-out locations. Again, this is a result which follows from the respective computational complexities of fitting estimates using these methods for held-out data, but this massive difference clearly demonstrates the value of alternative methods such as Algorithm 3.

# 5 Discussion

## 5.1 Conclusions

In this paper, we focus on developing a method for deriving estimates of a spatiotemporal multi-output Gaussian process by incorporating a Kalman filter to sequentially update the mean and covariance functions, as originally proposed by Todescato et al.. The extended version of their algorithm (Algorithm 3) appears to be empirically quite successful, showcasing all of the strengths of the original algorithm which the authors proposed while also being robust to certain relaxations of their assumptions, which is best exemplified by the strong performance of the temporal LMC kernel.

## 5.2 Limitations

One of the primary limitations of this work is that the simulations could be extended to a larger variety of data settings, and could be extended further to compare how the algorithms scale with higher-dimensional outputs. However, due to compute limitations, we were unable to run a suitable number of simulations with $D > 5$ outputs for the multi-output process. This limitation is somewhat ironic, as the primary goal of this investigation is to develop a method which can be implemented quicker and with less computational overhead than MOGP regression.

## 5.3 Future Work

The primary manner in which this work may be extended is to evaluate the performance of Algorithm 3 on some real-world datasets from fields of scientific inquiry such as ecology, environmental science or geostatistics. Assessing the performance of the proposed algorithm on real scientific measurements can provide feedback on how well the assumptions in this report may be met in a practical setting, and how effective this algorithm is at modelling real spatiotemporal dynamical systems. Furthermore, testing the scientific applications of the modelling algorithm may lead to new ideas, such as the adaptive location space method discussed by Todescato et al..

Another aspect in which this work can be extended is by increasing the complexity of the kernels and the data-generating process for the simulation study. In the simulations, we focused on observations coming from a pre-determined set $\mathcal{I}$ containing evenly-spaced points on a line segment. Future simulations whose input locations are generated in a more complex manner such as randomly-sampled points on a 2-D plane or surface or via a space-filling curve may yield different behaviour than the simulations whose inputs are conveniently evenly-spaced along a line.

Another area of continued algorithmic research related to this project is to examine the use of smoothing distributions such as the Rauch-Tung-Striebel (RTSS) smoother (Rauch et al., 1965; Särkkä and Svensson, 2023) or modified Bryson-Frazier smoother (Colemont et al., 2026). The primary use of these smoothing distributions is to develop estimates for the latent states $f(x_i, t_j)$ at *any* observation time $t_k$ instead of solely performing inference on the latent states corresponding to the latest-observed measurements. The RTSS is known Hartikainen and Särkkä (2010); Särkkä and Hartikainen (2012) to be closely related to temporal Gaussian processes, which can enable the use of smoothers for the multi-output Gaussian process problems considered in this work.

A similar task of future interest is to examine if the algorithms discussed in this report may be adjusted to work with dynamical systems whose transformations are governed by some unknown parameters. In this work, we have focused on dynamical systems whose underlying mechanics are

assumed to be fully understood. However, this is often a reductive and idealistic assumption in practice, and it may be improved upon by incorporating some unknown parameters into the state transition model, instead of assuming that the non-stochastic aspects of the dynamic evolution are fully known a priori. For example, a parametric or semi-parametric kernel function may have hyperparameters which can be inferred jointly with the latent states in the dynamical system using variational inference or empirical Bayes methods. Similarly, in certain applied contexts, it may be helpful investigate extensions of this method which incorporate change-point detection methods so that these methods are capable of modelling a larger class of spatiotemporal dynamical systems. Further developments regarding these aspects of the work may be particularly applicable to 'real-world' scientific research in fields such as epidemiology, ecology, environmental science, and control systems engineering, as the dynamical systems encountered in these research areas are often governed by time-varying behaviour, and we may lack complete knowledge of the corresponding mechanics driving the evolution of the system.

# References


Álvarez, M. A., Rosasco, L., and Lawrence, N. D. (2012). Kernels for Vector-Valued Functions: A Review. *Foundations and Trends in Machine Learning*, 4(3):195–266.

Bonilla, E., Chai, K., and Williams, C. (2007). Multi-task Gaussian Process Prediction. In Platt, J., Koller, D., Singer, Y., and Roweis, S., editors, *Advances in Neural Information Processing Systems*, volume 20. Curran Associates, Inc.

Boyle, P. and Frean, M. (2004). Dependent Gaussian Processes. In Saul, L., Weiss, Y., and Bottou, L., editors, *Advances in Neural Information Processing Systems*, volume 17. MIT Press.

Carron, A., Todescato, M., Carli, R., Schenato, L., and Pillonetto, G. (2016). Machine learning meets Kalman Filtering. In *2016 IEEE 55th Conference on Decision and Control (CDC)*, pages 4594–4599.

Colemont, T., Evens, B., Li, T. G.-F., and De Ceuster, F. (2026). Modified Bryson-Frazier Smoothing and Hyperparameter Learning for Temporal Gaussian Process Regression. In Karvonen, T., Bosch, N., Cockayne, J., Gessner, A., Hennig, P., and Kouw, W., editors, *Proceedings of the 2nd International Conference on Probabilistic Numerics*, volume 341 of *Proceedings of Machine Learning Research*, pages 25–38. PMLR.

Cressie, N. (1990). The origins of kriging. *Mathematical Geology*, 22(3):239 – 252.

Gardner, J. R., Pleiss, G., Bindel, D., Weinberger, K. Q., and Wilson, A. G. (2018). GPyTorch: Blackbox Matrix-Matrix Gaussian Process Inference with GPU Acceleration. In *Advances in Neural Information Processing Systems*.

Guhaniyogi, R. and Banerjee, S. (2018). Meta-Kriging: Scalable Bayesian Modeling and Inference for Massive Spatial Datasets. *Technometrics*, 60(4):430–444. PMID: 31007296.

Hartikainen, J. and Särkkä, S. (2010). Kalman filtering and smoothing solutions to temporal Gaussian process regression models. In *2010 IEEE International Workshop on Machine Learning for Signal Processing*, pages 379–384.

Huang, H., Blake, L. R., Katzfuss, M., and Hammerling, D. M. (2025). Nonstationary Spatial Modeling of Massive Global Satellite Data. *Journal of Computational and Graphical Statistics*, 34(4):1226–1239.

Kálmán, R. E. (1960). A New Approach to Linear Filtering and Prediction Problems. *Journal of Basic Engineering*, 82(1):35–45.

Katzfuss, M. and Guinness, J. (2021). A General Framework for Vecchia Approximations of Gaussian Processes. *Statistical Science*, 36(1):124 – 141.

Liu, H., Cai, J., and Ong, Y.-S. (2018). Remarks on multi-output Gaussian process regression. *Knowledge-Based Systems*, 144:102–121.

Pförtner, M., Wenger, J., Cockayne, J., and Hennig, P. (2025). Computation-Aware Kalman Filtering and Smoothing. In Li, Y., Mandt, S., Agrawal, S., and Khan, E., editors, *Proceedings of The 28th International Conference on Artificial Intelligence and Statistics*, volume 258 of *Proceedings of Machine Learning Research*, pages 2071–2079. PMLR.

Python Software Foundation (2024). Python 3.12.5. https://www.python.org/downloads/release/python-3125/. Released August 6, 2024.

Quiñonero-Candela, J. and Rasmussen, C. E. (2005). A Unifying View of Sparse Approximate Gaussian Process Regression. *Journal of Machine Learning Research*, 6(65):1939–1959.

Raitoharju, M. and Piché, R. (2019). On Computational Complexity Reduction Methods for Kalman Filter Extensions. *IEEE Aerospace and Electronic Systems Magazine*, 34(10):2–19.

Rasmussen, C. E. and Williams, C. K. I. (2006). *Gaussian Processes for Machine Learning*. The MIT Press.

Rauch, H. E., Tung, F., and Striebel, C. T. (1965). Maximum likelihood estimates of linear dynamic systems. *AIAA Journal*, 3(8):1445–1450.

Salari, A. and Zeinali, M. (2025). Kalman-Enhanced Dynamic Spatiotemporal Gaussian Process Regression for Real-Time Wind Turbine Power Estimation. In *2025 15th Smart Grid Conference (SGC)*, pages 1–4.

Särkkä, S. and Hartikainen, J. (2012). Infinite-Dimensional Kalman Filtering Approach to Spatio-Temporal Gaussian Process Regression. In Lawrence, N. D. and Girolami, M., editors, *Proceedings of the Fifteenth International Conference on Artificial Intelligence and Statistics*, volume 22 of *Proceedings of Machine Learning Research*, pages 993–1001, La Palma, Canary Islands. PMLR.

Särkkä, S. and Svensson, L. (2023). *Bayesian Filtering and Smoothing*. Institute of Mathematical Statistics Textbooks. Cambridge University Press, 2 edition.

Titsias, M. K. (2009). Variational Learning of Inducing Variables in Sparse Gaussian Processes. In van Dyk, D. and Welling, M., editors, *Proceedings of the Twelfth International Conference on Artificial Intelligence and Statistics*, volume 5 of *Proceedings of Machine Learning Research*, pages 567–574, Hilton Clearwater Beach Resort, Clearwater Beach, Florida USA. PMLR.

Todescato, M., Carron, A., Carli, R., Pillonetto, G., and Schenato, L. (2020). Efficient spatio-temporal Gaussian regression via Kalman filtering. *Automatica*, 118:109032.

# Appendix

## A AI Usage Statement

Two large language models – Claude Opus 5.5 (Anthropic) and GPT-5.6 Luna (OpenAI) – were used in the creation of this report.

Claude Opus 5.5 was employed for the creation and testing of Python code used for the simulations and replication of results from the prior paper. Claude Opus 5.5 was additionally used for assistance with some mathematical derivations and proofs within the written report. GPT-5.6 Luna was used for LaTeXtypesetting and formatting. Both large language models were used in searching for relevant background literature pertaining to this project.

All of the content in this document has been verified by the author, and is the sole responsibility of the author of this document. All AI-generated work in the creation of this project has been thoroughly reviewed and revised upon by the author, who is solely responsible for the content therein.

## B Code

For the sake of brevity, the code utilized in the creation of this project is not directly present in the appendix. All of the code used for this project is available in a public GitHub repository, located at https://github.com/hotz1/Dynamical-Multitask-Prediction. The `README.md` file in the root directory of the GitHub repository contains further information regarding these files.

# C Definitions and Technical Details

**Definition C.1** (Positive-Definite Matrix)**.** *A matrix* $\mathbf{M} \in \mathbb{R}^{n \times n}$ *is positive-definite (written* $\mathbf{M} \succ 0$*) if*

$$v^\mathsf{T}\mathbf{M}v > 0, \quad \text{for all } v \in \mathbb{R}^n \setminus \{0\} \tag{C.1}$$

*Similarly,* $\mathbf{M} \in \mathbb{R}^{n \times n}$ *is a positive-semidefinite (denoted* $\mathbf{M} \succeq 0$*) when the strict inequality above is loosened to* $v^\mathsf{T}\mathbf{M}v \geq 0$ *for all vectors* $v \in \mathbb{R}^n$*.*

**Definition C.2** (Gram Matrix)**.** *For a kernel function* $k : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$*, and inputs* $x_1, \ldots, x_n \in \mathcal{X}$*, the Gram matrix for the kernel function is given by*

$$K = \begin{bmatrix} k(x_1, x_1) & \cdots & k(x_1, x_n) \\ \vdots & \ddots & \vdots \\ k(x_n, x_1) & \cdots & k(x_n, x_n) \end{bmatrix} \tag{C.2}$$

**Definition C.3** (Positive-Definite Kernel)**.** *A function* $k : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ *is a positive-definite kernel function if for any finite set* $x_1, \ldots, x_n \in \mathcal{X}$ *and real coefficients* $c_1, \ldots, c_n \in \mathbb{R}$*,*

$$\sum_{i=1}^{n} \sum_{j=1}^{n} c_i c_j k(x_i, x_j) \geq 0 \tag{C.3}$$

*Equivalently, for any set of inputs* $x_1, \ldots, x_n \in \mathcal{X}$*, the resulting Gram matrix* $\mathbf{G} \in \mathbb{R}^{n \times n}$ *for the kernel* $k$ *is positive-semidefinite.*

**Definition C.4** (Cholesky Decomposition)**.** *The Cholesky decomposition of an* $n \times n$ *matrix* $\mathbf{M} \succeq 0$ *is given by* $\mathbf{M} = \mathbf{L}\mathbf{L}^\mathsf{T}$*, where* $\mathbf{L} \in \mathbb{R}^{n \times n}$ *is a real-valued lower-triangular matrix whose diagonal entries are all non-negative.*

**Definition C.5** (Reproducing Kernel Hilbert Space (RKHS))**.** *Let* $\mathcal{X}$ *be an indexing set, and let* $\mathcal{H}$ *be a Hilbert space of real-valued functions* $f : \mathcal{X} \to \mathbb{R}$ *equipped with the inner product* $\langle \cdot, \cdot \rangle_{\mathcal{H}}$ *and induced norm* $\|\cdot\|_{\mathcal{H}}$*.*

*For every* $x \in \mathcal{X}$*, define the evaluation functional* $L_x : \mathcal{H} \to \mathbb{R}$ *such that* $L_x[f] = f(x)$ *for all* $f \in \mathcal{H}$*.* $\mathcal{H}$ *is an RKHS over* $\mathcal{X}$ *if for all* $x \in \mathcal{X}$*, the evaluation functional* $L_x[f] := f(x)$ *is a bounded and continuous functional on* $\mathcal{H}$*.*

**Definition C.6** (Reproducing Kernel)**.** *Let* $\mathcal{H}$ *be an RKHS over* $\mathcal{X}$ *with* $\langle \cdot, \cdot \rangle_{\mathcal{H}}$ *and norm* $\|\cdot\|_{\mathcal{H}}$*. The function* $k : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ *is the reproducing kernel for* $\mathcal{H}$ *if it satisfies the reproducing property:*

$$f(x) = \langle f, k(x, \cdot) \rangle_{\mathcal{H}}, \quad \forall x \in \mathcal{X} \tag{C.4}$$

**Theorem C.1** (Representer Theorem)**.** *Suppose that we have some dataset* $\mathcal{D} := \{(x_i, y_i)\}_{i=1}^n$*.*

$$J[f] := \frac{\lambda}{2} \|f\|_{\mathcal{H}}^2 + \sum_{i=1}^{n} L\big(y_i, f(x_i)\big), \tag{C.5}$$

*where* $\lambda$ *is a regularization parameter, and* $L$ *is some loss function. Let* $\mathcal{H}_k$ *be an RKHS with reproducing kernel* $k$*. The Representer Theorem states that the minimizer* $f^*(x)$ *satisfies*

$$f^*(x) = \operatorname*{arg\,min}_{f \in \mathcal{H}_k} J[f] = \sum_{i=1}^{n} \alpha_i k(x, x_i) \tag{C.6}$$

**Definition C.7** (Power Spectral Density)**.** *The power spectral density (PSD) of a univariate real-valued function $z(\tau)$ is given by*

$$S(\omega) = \int z(\tau) e^{-2\pi i \omega \tau} \mathrm{d}\tau \tag{C.7}$$

**Theorem C.2** (Wiener-Khinchin Theorem)**.** *Suppose that $S(\omega)$ is the power spectral density of a univariate real-valued function $z(\tau)$, where $z(\tau)$ is the covariance of a stationary Gaussian process with $\tau := t - t'$. The PSD $S(\omega)$ is equal to the Fourier transform of the covariance $z$, given by*

$$S(\omega) := \mathcal{F}[z(\tau)](\omega) \tag{C.8}$$

**Theorem C.3** (Spectral Factorization)**.** *Suppose that $S(\omega)$ is the power spectral density of a univariate real-valued function $z(\tau)$, and $S(\omega)$ is a rational function of order $2r$. The power spectral density $S(\omega)$ can be decomposed into $S(\omega) = W(\boldsymbol{i}\omega)W(-\boldsymbol{i}\omega)$, where $W(\cdot)$ is described by the following:*

$$W(\boldsymbol{i}\omega) := \frac{b_{r-1}(\boldsymbol{i}\omega)^{r-1} + b_{r-2}(\boldsymbol{i}\omega)^{r-2} + \cdots + b_0}{(\boldsymbol{i}\omega)^{r} + a_{r-1}(\boldsymbol{i}\omega)^{r-1} + \cdots + a_0} \tag{C.9}$$

**Definition C.8** (Stationary Kernel)**.** *A kernel function $k : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ is a stationary kernel if the function solely depends on the difference of its inputs. Formally, every stationary kernel is a function of the form*

$$k(x, x') = \varphi(\|x - x'\|), \tag{C.10}$$

*where $\varphi(\cdot)$ is some univariate real-valued function.*

**Definition C.9** (Isotropic Kernel)**.** *A kernel function $k : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ is an isotropic kernel if the function solely depends on some normed distance between the inputs. Formally, every isotropic kernel is a function of the form*

$$k(x, x') = \varphi(\|x - x'\|), \tag{C.11}$$

*where $\varphi(\cdot)$ is some univariate real-valued function.*

**Definition C.10** (Common Isotropic Kernels)**.** *Some isotropic kernels Definition C.9 which are commonly used in spatial covariance modelling are the following:*

$$\textbf{Matérn(1/2):} \quad C_{1/2}(x, x') = \sigma^2 \exp\Big(-\frac{d}{\ell}\Big) \tag{C.12a}$$

$$\textbf{Matérn(3/2):} \quad C_{3/2}(x, x') = \sigma^2 \Big(1 + \frac{\sqrt{3}d}{\ell}\Big) \exp\Big(-\frac{\sqrt{3}d}{\ell}\Big) \tag{C.12b}$$

$$\textbf{Matérn(5/2):} \quad C_{5/2}(x, x') = \sigma^2 \Big(1 + \frac{\sqrt{5}d}{\ell} + \frac{5d^2}{3\ell^2}\Big) \exp\Big(-\frac{\sqrt{5}d}{\ell}\Big) \tag{C.12c}$$

$$\textbf{Gaussian (RBF):} \quad K_{RBF}(x, x') = \sigma^2 \exp\Big(-\frac{d^2}{2\ell^2}\Big) \tag{C.12d}$$

*In each of the isotropic kernel examples shown above, we define $d := \|x - x'\|$. The hyperparameters $\sigma^2$ and $\ell$ respectively represent scales for the distance and the kernel output, allowing greater flexibility in models using these isotropic kernel functions.*

# D Figures

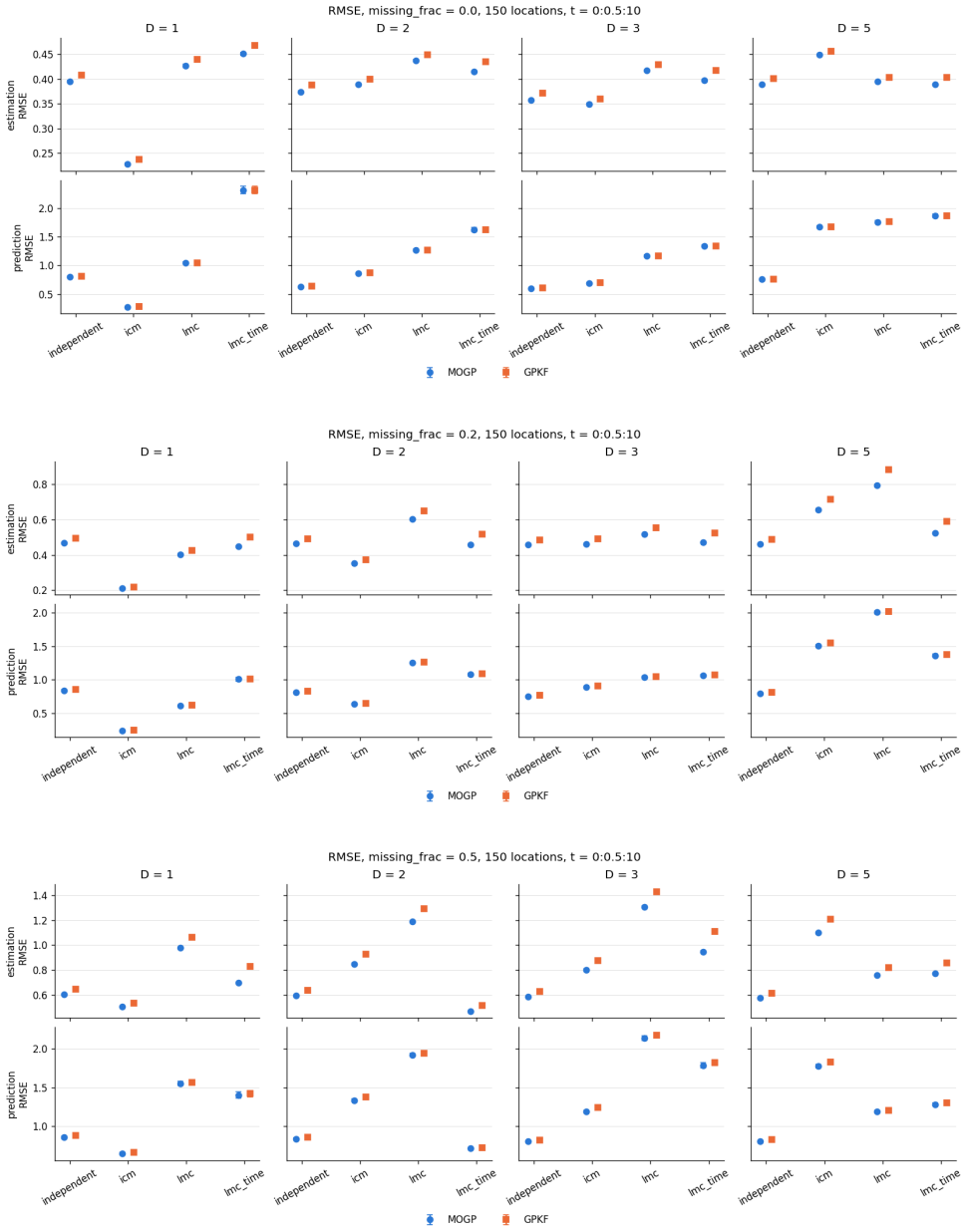



Figure 1: Plots comparing the RMSE for the two estimation methods (MOGP regression in blue, Algorithm 3 in orange), stratified by kernel type and missingness proportion $\rho$.

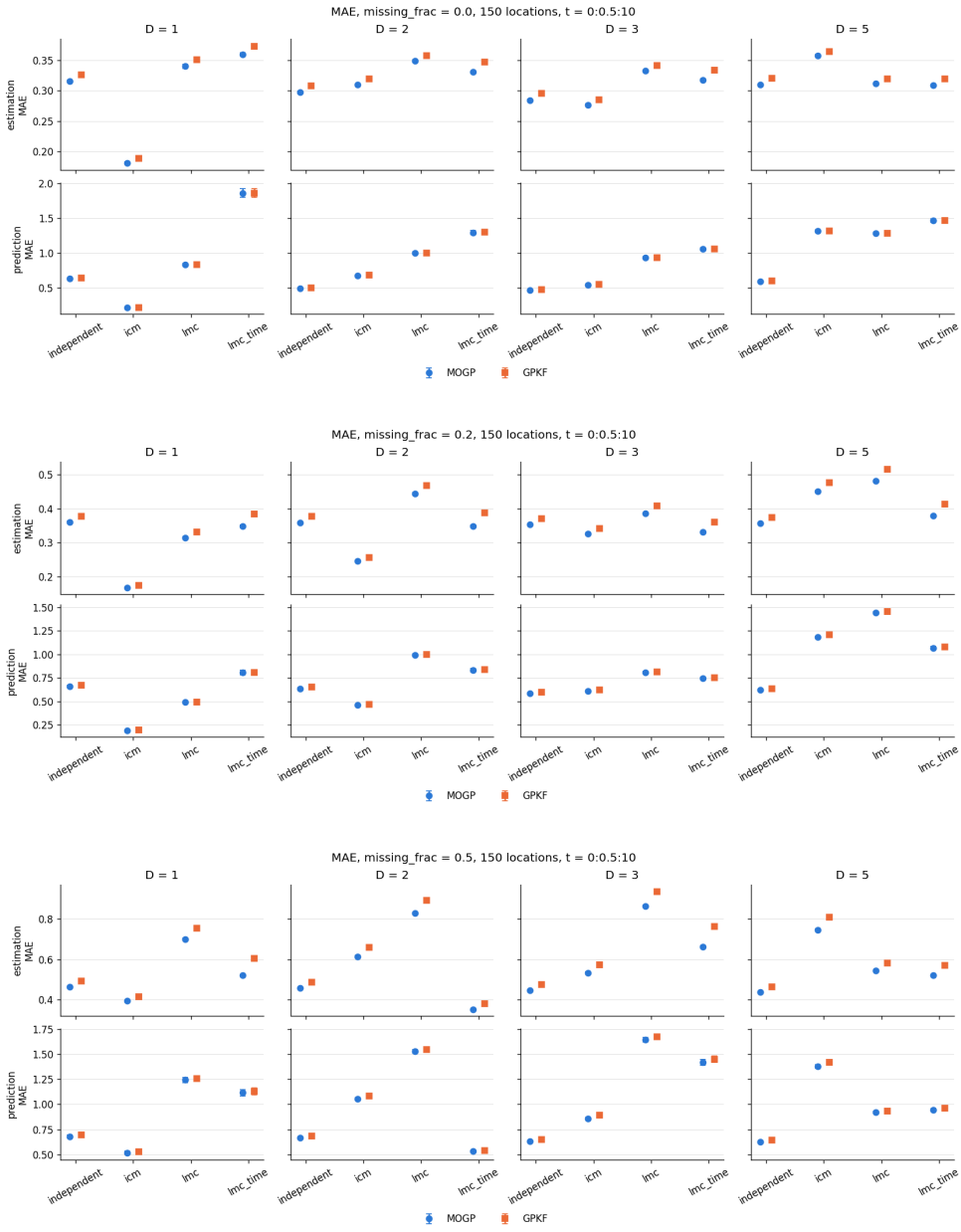



Figure 2: Plots comparing the MAE for the two estimation methods (MOGP regression in blue, Algorithm 3 in orange), stratified by kernel type and missingness proportion $\rho$.

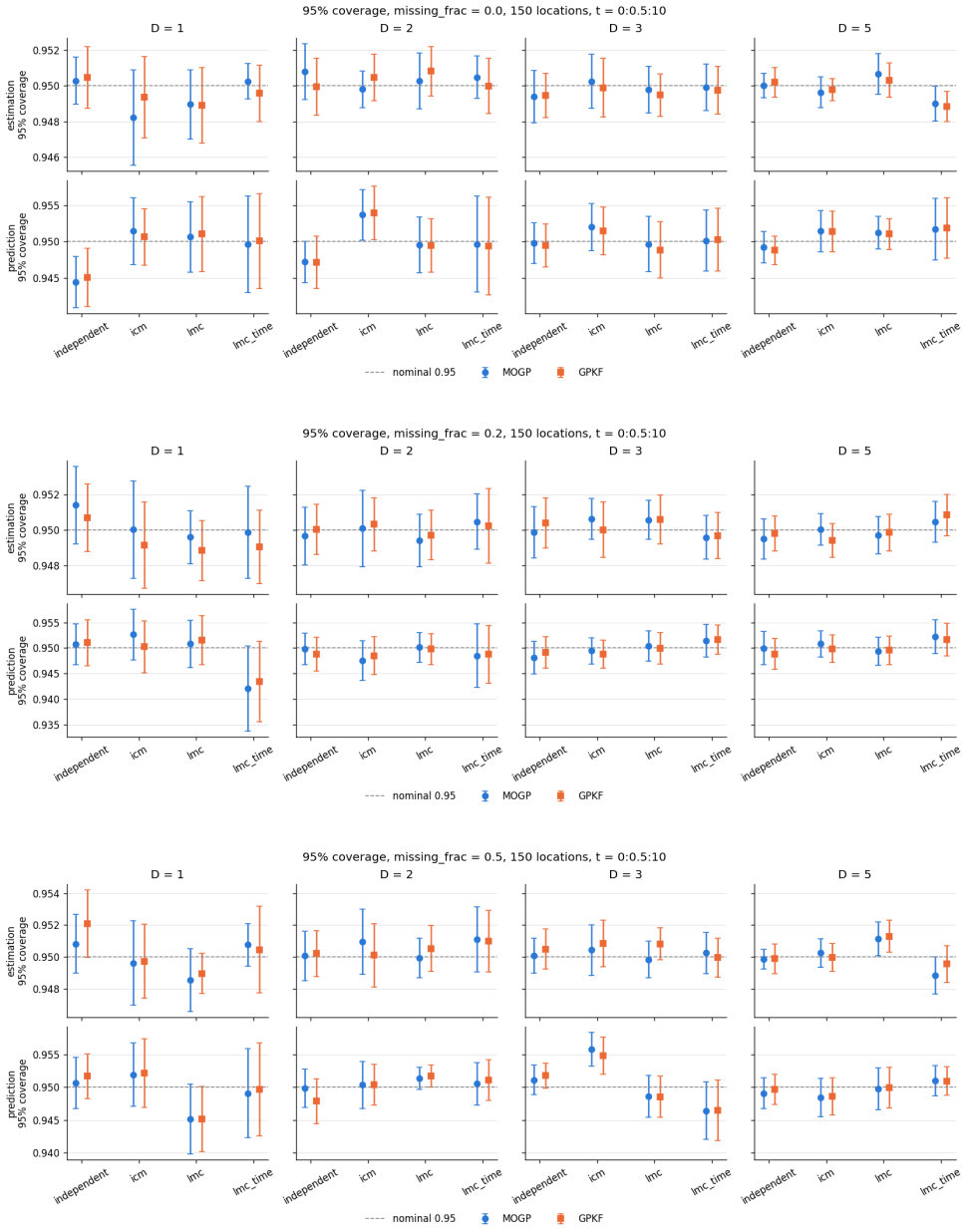



Figure 3: Plots comparing the coverage proportions of the 95% credible intervals, stratified by kernel type and missingness proportion $\rho$.

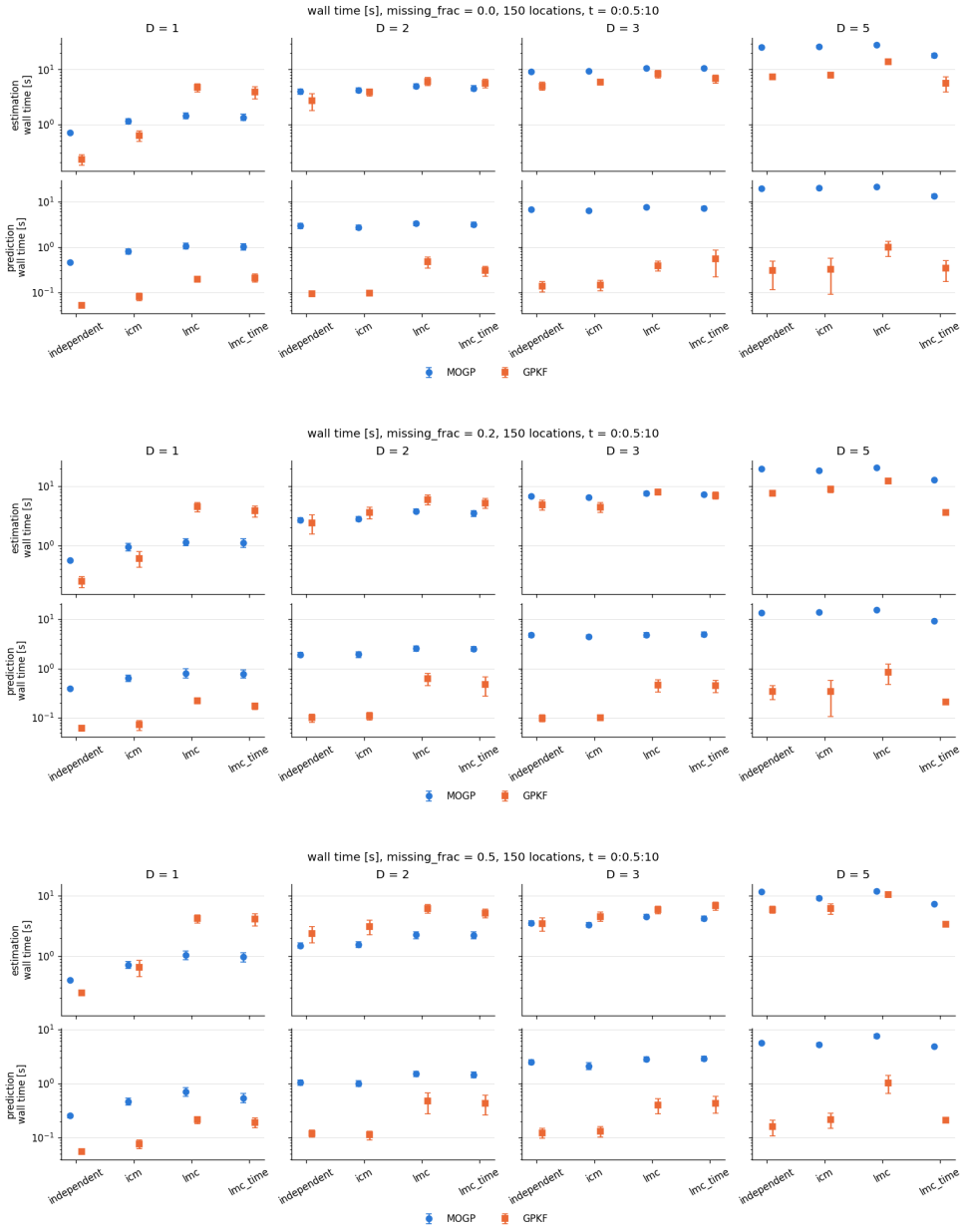



Figure 4: Plots comparing the wall-clock time for the two algorithms of interest, stratified by the kernel type and missingness proportion $\rho$.